

POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Software Engineering 2 - RASD

Author: **Simone Controguerra**

Author: **Matteo Delogu**

Author: **Gianluca Di Paola**

Advisor: Prof. Elisabetta Di Nitto
Academic Year: 2025-2026

Contents

Contents **i**

1 Introduction **1**
1.1 Purpose . . . 1
1.2 Scope . . . 1
1.2.1 Goals . . . 1
1.2.2 Phenomena Table . . . 2
1.3 Definitions, acronyms and abbreviations . . . 5
1.4 Revision history . . . 6
1.5 Reference documents . . . 6
1.6 Document structure . . . 6

2 Overall Description **9**
2.1 Product perspective . . . 9
2.2 Scenarios . . . 9
2.2.1 Domain class diagram . . . 13
2.3 Product functions . . . 14
2.3.1 Account Lifecycle . . . 14
2.3.2 Trip: Recording (Automatically or Manually) . . . 14
2.3.3 Trip: Review and Publishing . . . 15
2.3.4 Trip: Weather Enrichment . . . 15
2.3.5 Bike-Path: Manual Creation . . . 16
2.3.6 Bike-Path: Automatic Generation . . . 16
2.3.7 External API: Street Suitability . . . 17
2.3.8 Query: from Origin to Destination . . . 17
2.3.9 Merging and Change Propagation . . . 17
2.4 User characteristics . . . 18
2.5 Assumptions, dependencies and constraints . . . 19

3 Specific Requirements **21**
3.1 External Interface Requirements . . . 21
3.1.1 User Interfaces . . . 21
3.1.2 Hardware Interfaces . . . 29
3.1.3 Software Interfaces . . . 30
3.1.4 Communication interfaces . . . 30
3.2 Functional Requirements . . . 31
3.2.1 Requirements Mapping . . . 32
3.2.2 Use case diagram . . . 42
3.2.3 Use Cases . . . 43
3.3 Performance Requirements . . . 59
3.3.1 Client . . . 59
3.3.2 Server . . . 59
3.4 Design Constraints . . . 60
3.4.1 Standards compliance: . . . 60
3.4.2 Hardware limitations . . . 60
3.4.3 Any other constraint . . . 60
3.5 Software system attributes . . . 61
3.5.1 Reliability . . . 61
3.5.2 Availability . . . 61
3.5.3 Security . . . 61
3.5.4 Maintainability . . . 62
3.5.5 Portability . . . 62

4 Formal Analysis using Alloy **63**
4.1 Examples . . . 68
4.1.1 Single report determines street status and path score . . . 69
4.1.2 Freshness wins over older majority of reports . . . 70
4.1.3 Majority voting wins among reports with the same freshness . . . 71

5 Effort spent **73**

6 References **75**

1 | Introduction

1.1. Purpose

Nowadays self-care activities have become really popular such as keeping themselves fit, in particular a lot of people ride their bike either to stay healthy or to adopt an eco-friendly means of transport. However, streets are not always the safest place for bikes since cars can run very fast there. The solution is to ride on designated bike paths or safe roads where cars are few and they cannot run too fast.

Best Bike Paths is a service that offers an innovative solution to bikers that want to ride safely. Its purpose is whether to give the opportunity to its users to record their personal trips and store them as an archive of sport activities, enriched with different kinds of related information, or to provide suggestions for safe and efficient bike paths to reach a given destination, highlighting the better ones. Best Bike Paths is a service with community features, that aims to build a network of bike paths for everyone.

1.2. Scope

Best Bike Paths is a service where only registered users can store their bike trips and they can see the related metrics and weather conditions. Moreover, they can suggest new bike paths indicating also the status of the streets and eventual obstacles. Bike trips are private and personal, but it's possible to use the collected data during the activity to create a new bike path that will be published and available for everyone, also not registered users, that are searching for the best path to reach a desired destination. The continuous monitoring of streets' status and their obstacles, by the different registered users, allows the platform to be always updated with fresh data.

1.2.1. Goals

- **[G1]** Enable the cycling community to access a shared catalog of safe and reliable bike paths.

- **[G2]** Allow registered users to store trips with automatically-collected sensor data and, if available, associated weather.
- **[G3]** Develop and maintain a collaborative inventory of bike paths and their score, along with status and obstacle information regarding the streets they are composed of.
- **[G4]** Enable registered users to contribute to the enrichment of the bike paths catalog, by publishing automatically-recorded bike paths.
- **[G5]** Enable registered users to contribute to the enrichment of the bike paths catalog, by publishing manually-entered bike paths.
- **[G6]** Allow any user (registered or unregistered) to search for bike paths from Origin to Destination, ordered by their effectiveness for such parameters.
- **[G7]** Ensure bike paths consist only of roads suitable and safe for cycling.
- **[G8]** Reduce errors and false positives through user confirmation and information merging.
- **[G9]** Support a secure account lifecycle for registered users, including authentication, recovery, consent capture, and preferences management.

1.2.2. Phenomena Table

Code	Phenomenon	Who controls it?	Shared?
[P1]	Any user (registered or not) follow a new bike path.	World	No
[P2]	A registered user starts cycling.	World	No
[P3]	Road status (needs maintenance, optimal, mediocre, ...).	World	No
[P4]	A new user creates a profile.	World	Yes
[P5]	A registered user starts a trip session.	World	Yes
[P6]	A registered user manually defines (in terms of streets, status, etc.) and publishes a bike path.	World	Yes
[P7]	A registered user confirms or corrects information acquired by the system and publishes the autogenerated bike path.	World	Yes

Code	Phenomenon	Who controls it	Shared?
[P8]	Any user (registered or not) requires a bike path by specifying origin and destination.	World	Yes
[P9]	Device sensors (GPS, accelerometer, gyroscope) acquire data.	World	Yes
[P10]	The weather-service responds to the request.	World	Yes
[P11]	The navigation-system service responds to the requests.	World	Yes
[P12]	The system visually notifies the user that the trip recording has started.	Machine	Yes
[P13]	The system provides all recorded trip metrics.	Machine	Yes
[P14]	The system notifies the registered user that an autogenerated bike path is ready for detection revision.	Machine	Yes
[P15]	The system shows to any user (registered or not) all the possible bike paths, ordered by overall score, to accomplish their request.	Machine	Yes
[P16]	The system prevents registered users from publishing bike paths with unallowed streets.	Machine	Yes
[P17]	The system updates a bike path, inserting the new re-computed score, whenever the status of a street changes.	Machine	Yes
[P18]	The system calls the weather-forecast service to add weather info to a trip.	Machine	Yes
[P19]	The system calls the navigation-system service to evaluate road safety for bikers (elements taken into account are of the likings of: proper bike track, car presence frequency, average speed of a bike, etc.).	Machine	Yes

Code	Phenomenon	Who controls it	Shared?
[P20]	The system detects when the registered user is cycling and processes relevant data, to obtain performance metrics, to detect obstacles and such.	Machine	No
[P21]	The system calculates the overall score of a bike path, depending on its road status and effectiveness (in terms of allowing any user to reach the desired destination from a given origin).	Machine	No
[P22]	The system re-computes the score of a bike path whenever the status of a street changes.	Machine	No
[P23]	The system stores a new bike path.	Machine	No

Table 1.1: Phenomena

1.3. Definitions, acronyms and abbreviations

- **BBP (Best Bike Paths):** The name of the system.
- **Bike Path:** A route specifically designated or suitable for bicycle travel, which can be either manually defined by users or automatically composed by the system. It is formally defined as a set consisting of: a Starting Point, a Finishing Point, and an ordered sequence of traversable Streets (see definition of Street).
- **Trip:** A standalone, recorded instance of a Registered User traveling from one point to another, which includes sensor data, metrics, and potentially weather information.
- **Street:** A segment of road suitable for cycling (factors taken into account include average bike speed VS road speed limit, roadway width, traffic volume, dedicated bike lanes, etc.) that forms part of a bike path, with attributes such as status and presence of irregularities (e.g., potholes).
- **Navigation System:** An external service or API used by the system to evaluate road safety for bikers, considering factors such as bike track availability and traffic conditions.
- **Weather-forecast System:** An external service or API used by the system to provide weather information associated with a trip.
- **GPS (Global Positioning System):** A satellite-based navigation system that provides location and time information, in all weather conditions, anywhere on Earth.
- **IMU (Inertial Measurement Unit) Sensors:** Electronic devices that measure and report a physical object's specific force, angular rate and orientation, using a combination of both accelerometer and gyroscope.
- **API (Application Programming Interface):** A set of protocols and tools that enable different software applications to communicate with each other.
- **JSON (JavaScript Object Notation):** A lightweight data-interchange format that is easily readable and writable by both humans and machines.
- **UI (User Interface):** The interface through which users interact with the platform, encompassing buttons, screens, and navigation elements.
- **TTL (time-to-live):** Maximum duration for a cache entry to be considered valid.

- **Redundancy:** The inclusion of supplementary components or systems to ensure reliability and availability in the event of a primary component failure.
- **Scalability:** The system's capacity to manage a growing number of users or increased load by incorporating additional resources without significant degradation in performance.
- **Uptime:** The period during which a system is fully operational and accessible to users without interruption.
- **Backup:** The process of duplicating and archiving data to ensure restorability in the event of loss or damage to the original data.

1.4. Revision history

- **v1.0** First version of the document, revised on 22th of December 2025.

1.5. Reference documents

Professor Elisabetta Di Nitto, *Slides of the course "Software Engineering 2"* at PoliMi. Polytechnic University of Milan, *Specifications of the RASD assignment for the Software Engineering II course.*

1.6. Document structure

1. **Introduction:** it provides an overview about the project, focusing on its goals, purposes, and phenomena.
2. **Overall Description:** it includes a more detailed view focused on the high-level description of system's operations and on the illustration of world and machine's phenomena.
3. **Specific Requirements:** it shows an analysis of the requirements and more information regarding the use cases, the hardware and software limitations, the different interfaces and their requirements.
4. **Formal Analysis using Alloy:** it includes a formal representation of world phenomena using Alloy.
5. **Effort spent:** it indicates how the workload has been distributed among the contributors.

6. **References:** it includes references to documents and tools that have been used in the writing of the document.

2 | Overall Description

2.1. Product perspective

2.2. Scenarios

Scenario 1: User Registration

A new user downloads BBP and wants access to all the functionalities of the service. They provide email, password and accept the terms of service. The system validates the email, creates an account and requests consent for sensor's data use, location and data collection. After confirmation from the user, the service stores their preferences. Finally, the user becomes "registered" and can use both personal and collaborative features.

- Phenomena covered: P4.

Scenario 2: Trip Recording

The user starts cycling. The system either detects such activity (the system infers a cycling state based on sensor patterns) or is alerted manually by the user; then, it starts collecting data from the device's sensors (GPS, accelerometer, gyroscope), in order to obtain performance metrics (e.g., average speed during the trip); the recording of data proceeds also in the case of network disruptions. When the user is finished, the system stops the recording, calculates trip metrics (e.g., distance, duration, average speed) and shows a summary.

- Phenomena covered: P2, P5, P9, P12, P13, P20.

Scenario 3: Bike path: Manual Mode

A registered user creates a new bike path by entering start, finish, and the ordered streets. For each street, they specify the status (i.e., the overall condition) and irregularities (e.g., potholes). The system validates streets (checking for compliance) and stores the new path as public data.

- Phenomena covered: P6, P16, P23.

Scenario 4: Bike path: Automatic Mode

The user starts cycling; the system detects it thanks to the device's sensors (GPS, accelerometer, gyroscope). While the user cycles, the system acquires data from the sensors and generates a candidate bike path. After the ride, the system shows the user all detected obstacles. The user reviews and confirms or corrects them, then approves publication. The system updates the database's street data, based on information freshness and overall number of confirmations.

- Phenomena covered: P7, P9, P14, P20, P23.

Scenario 5: Merging of information

Several users travel the same street on different days. Each one signals the presence or contradicts the presence of potholes (data). The system updates the street's status (i.e., overall condition) based on freshness and number of confirmations, then updates the score of any path including that street.

- Phenomena covered: P17, P21, P22.

Scenario 6: Bike Path Query from Origin to Destination

A user of any kind (registered or not) specifies Origin and Destination. The system retrieves all available bike paths, ranks them by overall score and shows them to the user, ordered by efficiency.

- Phenomena covered: P8, P15, P21.

Scenario 7: Street Safety Validation

When a user creates a bike path, the system calls the external API of a navigation service to check for road compliance, in respect to safety features (average bike speed limit, roadway width, traffic volume, dedicated bike lanes, etc.). If any street is found unallowed, the publication process is interrupted until any compliant modifications are made.

- Phenomena covered: P11, P16, P19.

Scenario 8: Street Status Update and Bike Path Score Recomputing

When a street's condition changes and a user reports it, the system updates the status of the street and then recomputes the score of all bike paths including that street. Recomputed scores affect Origin-Destination ordering for the bike path query.

- Phenomena covered: P17, P21, P22.

Scenario 9: Trip: Weather Data Association

After trip recording, the system calls a weather API for data concerning the trip's time interval: the system then receives weather information (e.g, temperature, wind) and links it to the trip.

- Phenomena covered: P10, P18.

Scenario 10: User Account Deletion

A user chooses to leave the service by deleting their account. The system confirms the request and removes personal data according to the terms of service, while keeping published bike paths. A confirmation message is shown.

- Phenomena covered: None specifically.

Coverage: Scenarios VS Phenomena

	P1	P2	P3	P4	P5	P6	P7	P8	P9	P10	P11	P12	P13	P14	P15	P16	P17	P18	P19	P20	P21	P22	P23
S1				✓																			
S2		✓			✓				✓			✓	✓							✓			
S3						✓										✓							✓
S4							✓		✓					✓						✓			✓
S5																	✓				✓	✓	
S6								✓							✓						✓		
S7											✓					✓			✓				
S8																	✓				✓	✓	
S9										✓								✓					
S10																							

Table 2.1: Scenario × Phenomena Coverage Matrix

2.2.1. Domain class diagram

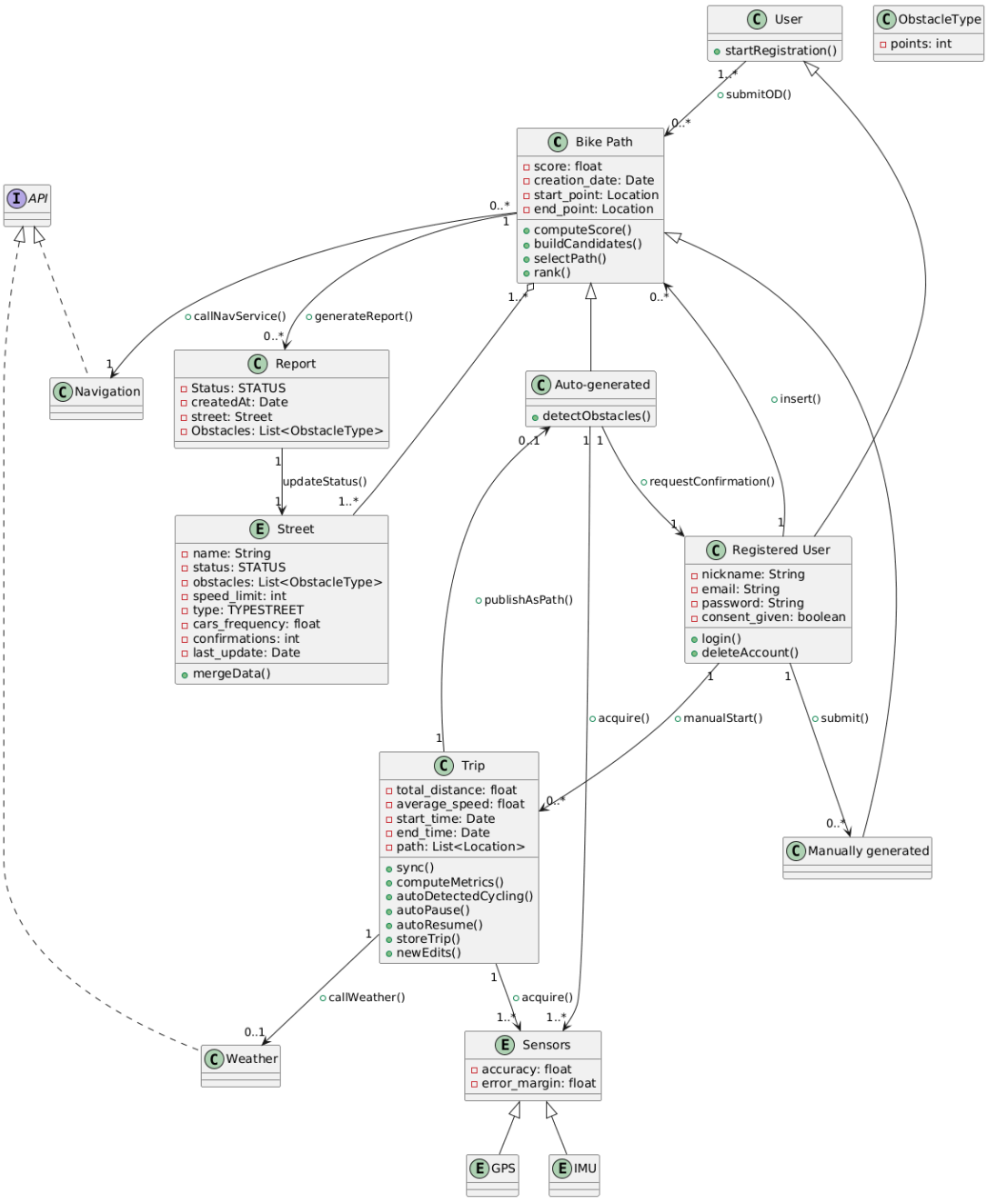


Figure 2.1: Class Diagram

2.3. Product functions

2.3.1. Account Lifecycle

Allows a user to change from Unregistered to Registered by completing the registration process, providing necessary information (e.g., email, password) and accepting the terms of service. Immediately after registration, the system requests GPS and IMU Consent. Lack of consent restricts features to non-sensing operations; consent can be granted or revoked later. Accounts can be deleted at any time.

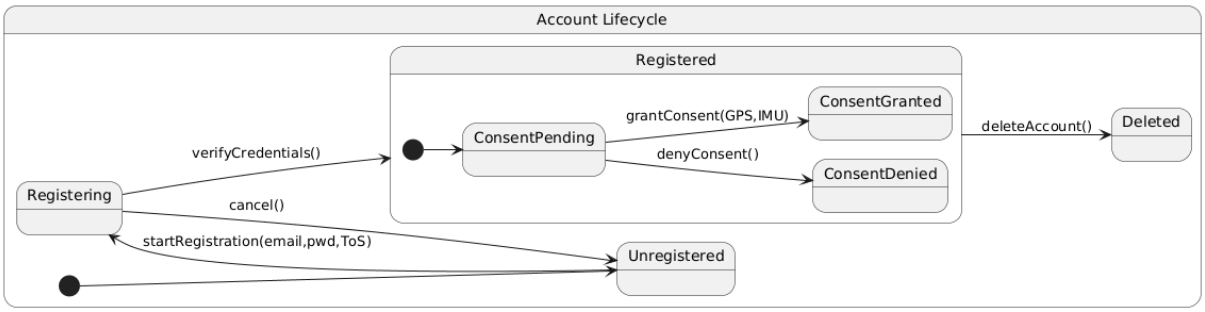


Figure 2.2: Account Lifecycle: Registration and Consent

2.3.2. Trip: Recording (Automatically or Manually)

Enables the system to record trips either through auto-detection (GPS/IMU) or manual initiation. During recording, a loss of motion automatically pauses the recording (users can also manually pause); then, the activity can be resumed, either automatically or manually. Upon stopping, the system computes metrics and saves the trip. Synchronization occurs when connectivity is available.

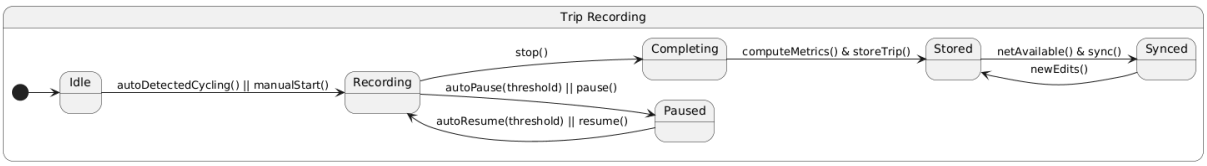


Figure 2.3: Trip Recording: Auto and Manual Modes

2.3.3. Trip: Review and Publishing

Allows the user to review a recorded trip summary and annotate onto it. The trip remains private unless the user explicitly chooses to publish it online, transforming it into a bike path; in such case, the system needs to validate the candidate bike path generated from the trip, therefore publishing or rejecting it.

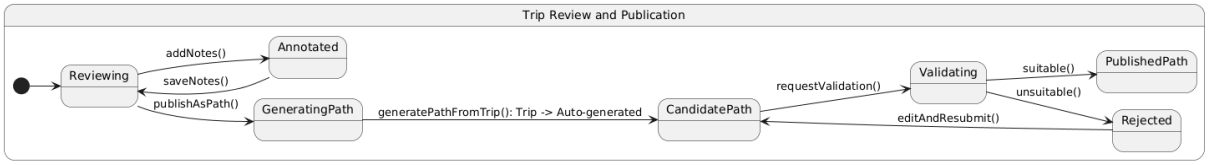


Figure 2.4: Trip Review and Publishing: From Private Trip to Public Bike Path

2.3.4. Trip: Weather Enrichment

Enhances a stored trip with weather data. The system queries the weather service. If successful, weather data related to the trip's timespan is attached to the trip. If the query fails, the system retries up to a policy-defined limit; if still unsuccessful, the trip will remain valid but without any weather information attached.

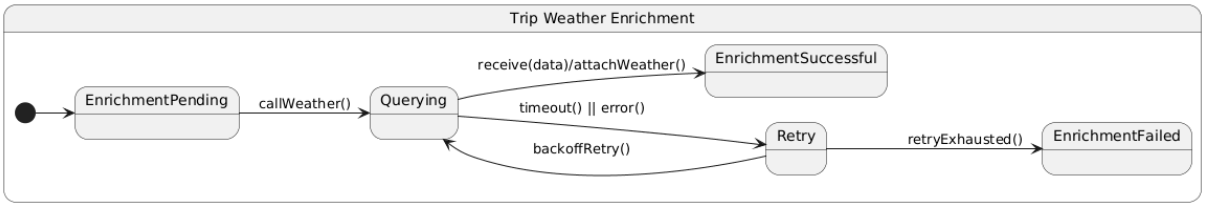


Figure 2.5: Trip Weather Enrichment: Adding Environmental Context

2.3.5. Bike-Path: Manual Creation

Enables the user to draft a bike path, specifying: starting point, finishing point and an ordered sequence of traversable streets, along with their status and the presence of any irregularities. Upon submission, the system validates the streets and checks the bike path for correctness and compliance. If valid, it is approved and then published; otherwise, it is rejected with optional specified reasons.

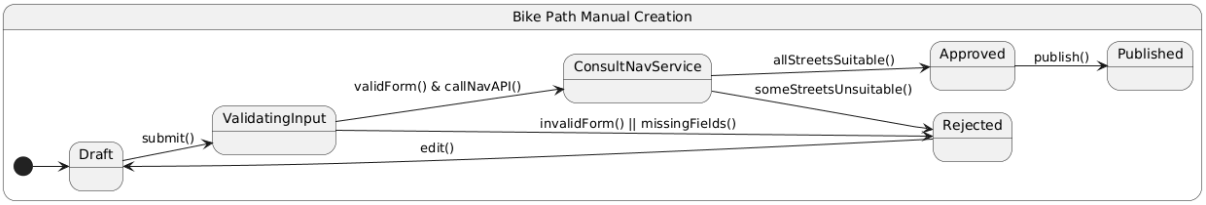


Figure 2.6: Bike Path Manual Creation: User-Defined Paths

2.3.6. Bike-Path: Automatic Generation

Automatically generates a candidate bike path from a recorded trace obtained from GPS/IMU data processing. The user enters the review mode, confirming and/or correcting detected obstacles and street selection. Upon submission, the system validates the streets and checks the bike path for correctness and compliance. If valid, it is approved and then published; otherwise, it is rejected with optional reasons.

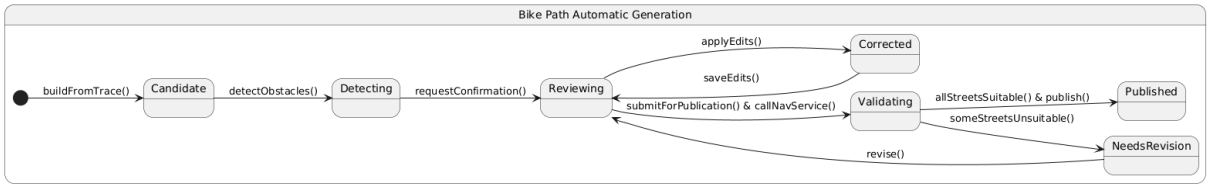


Figure 2.7: Bike Path Automatic Generation: From Recorded Trace to Bike Path

2.3.7. External API: Street Suitability

Submits all streets to a navigation system external API for any pending publication of a bike path, based on which each street is marked valid or invalid.

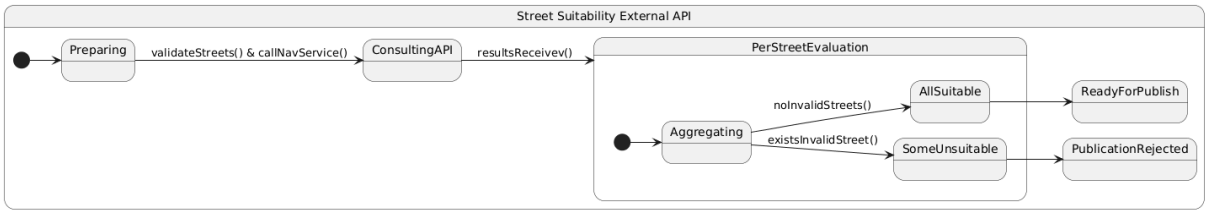


Figure 2.8: External API: Validating Street Suitability

2.3.8. Query: from Origin to Destination

Allows the user to issue an Origin-to-Destination request. The system composes a list of candidate bike paths, retrieves their scores, and presents the results, containing all the suitable bike paths ranked by the highest score. Selecting a result opens the details page of such bike path. After an update on the status of a street, rankings shall be recomputed.

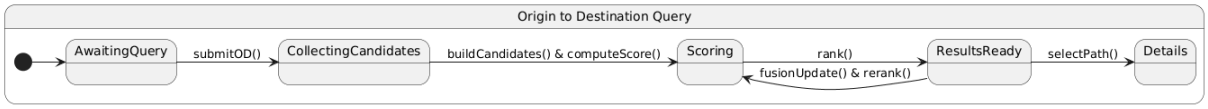


Figure 2.9: Query: Finding the Best Bike Path from A to B

2.3.9. Merging and Change Propagation

Accumulates contributions (i.e., status and presence of irregularities) from users to streets. The system merges information based on freshness and overall number of confirmations. When the street is merged, dependent bike paths (i.e., the ones containing that street) scores are recomputed according to the new information.

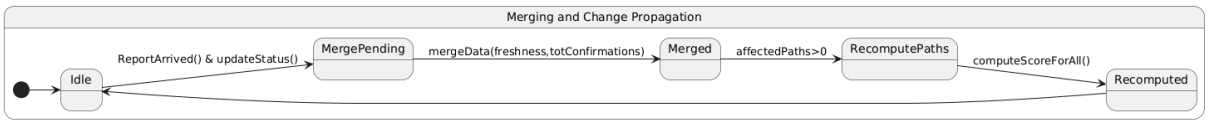


Figure 2.10: Merging and Change Propagation: Accumulating User Contributions

2.4. User characteristics

The system recognizes two kinds of users: Unregistered User and Registered User.

Unregistered users can query the catalog by specifying Origin and Destination and then view all available bike paths, ordered by overall score; they do not contribute data nor access personal trip features.

Registered users, which are presumed to own a mobile device with GPS and IMU Sensors, can:

(i) record personal trips (automatic start/stop, offline capture, metrics), optionally enriched with weather;

(ii) create bike paths manually by entering start, finish and an ordered street sequence with perceived status/irregularities;

(iii) review and confirm or correct obstacles detected from their automatic recordings before publication;

(iv) publish approved content to the shared catalog, where the system verifies street compliance via the navigation API and then fuses contributions at street level, propagating updates to all affected paths;

(v) manage account settings (consents, profile, deletion).

2.5. Assumptions, dependencies and constraints

In order to satisfy all the goals, the system requires the following assumptions to be met:

- **[A1] Compliance with guidelines:** All users (registered and unregistered) are expected to follow the platform's rules and guidelines and provide accurate, up-to-date information.
- **[A2] Valid credentials:** To use the service, registered users are expected to have a valid email address.
- **[A3] User's mobile device:** The user must possess a mobile device (e.g., smartphone) with integrated sensors (mandatory: GPS, accelerometer, gyroscope) capable of internet communication and application processing.
- **[A4] Sensor reliability:** GPS, accelerometer, gyroscope, and other necessary sensors are available, functioning correctly, and calibrated on the user's device.
- **[A5] GPS data detection:** The error margin of the GPS signal shall be within acceptable thresholds to allow for the detection of the starting point (origin) and arrival point (destination).
- **[A6] Accelerometer and Gyroscope sensitivity:** Data detection from accelerometer and gyroscope sensors must be accurate enough to correctly detect potholes and other street irregularities.

In addition, the systems requires some certain dependencies:

- **[D1] Reliable internet access:** All users (registered and unregistered) shall have a stable internet connection to perform certain actions.
- **[D2] Weather-forecast Service API:** The weather forecast service API must be correctly implemented, sufficiently available, and functioning correctly.
- **[D3] Navigation-system Service API:** The navigation system service API must be correctly implemented, sufficiently available, and functioning correctly.

3 | Specific Requirements

3.1. External Interface Requirements

3.1.1. User Interfaces

This section describes briefly the main screens and interactions presented to the user. We also include mockups to illustrate the intended design. We kept ease of use as main objective while creating the design, exploiting also mobile devices touch screens capabilities. Even if the UI is immediate to understand, an "On boarding" phase should be considered when developing the final product.

Look and feel can vary depending on the implementation

In this section we will concentrate on many critical interfaces, that are key to deliver a good final product. The interfaces described are: `Home with current trip`, `path update`, `Bike Path Search & Results`, `Manual Path Insertion` and `past trip recordings`. We also clarify the behavior of the `Login & Sign Up` page, since sign-in is optional and therefore the user must make an explicit choice at first launch.

Login & Sign Up

When the application opens for the first time, the user is presented with the Login page. It allows registered users to authenticate and access their set of features, it also makes clear that searching and viewing bike paths (public feature) is available even without creating an account. The screen offers standard credential fields for email and password, as well as an optional third-party single sign on (SSO) method.

On first authentication, the application shall request explicit consent for background data recording (location and activity) via an '*I understand and agree*' acknowledgment. Until consent is granted, background recording shall remain disabled.

If the user chooses to sign in and the credentials are correct, the application transitions directly to the `Home` screen. If authentication fails, the page remains visible and displays

a concise message that explains what went wrong (e.g., invalid credentials or a temporary network issue).

The login page will be presented again, at app startup, only to users not logged in

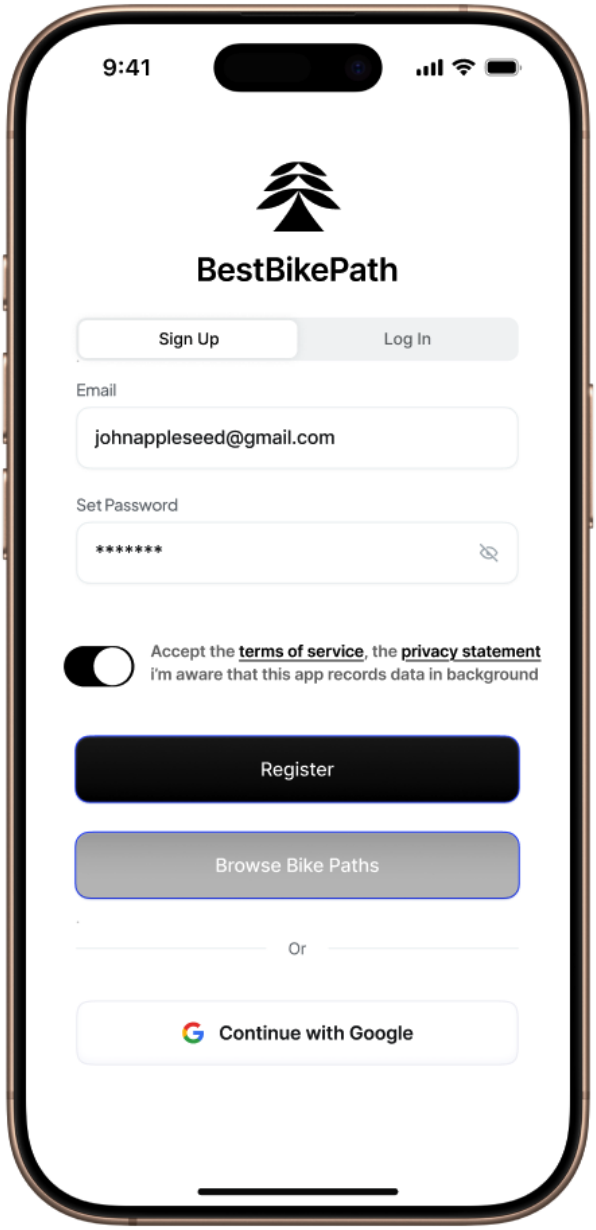


Figure 3.1: Possible Login UI

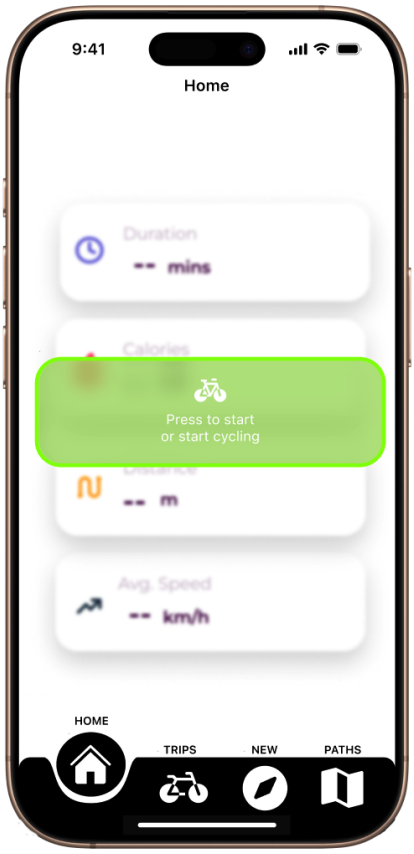
Home

When the application opens and no trip is currently active, the Home acts as a welcoming starting point.

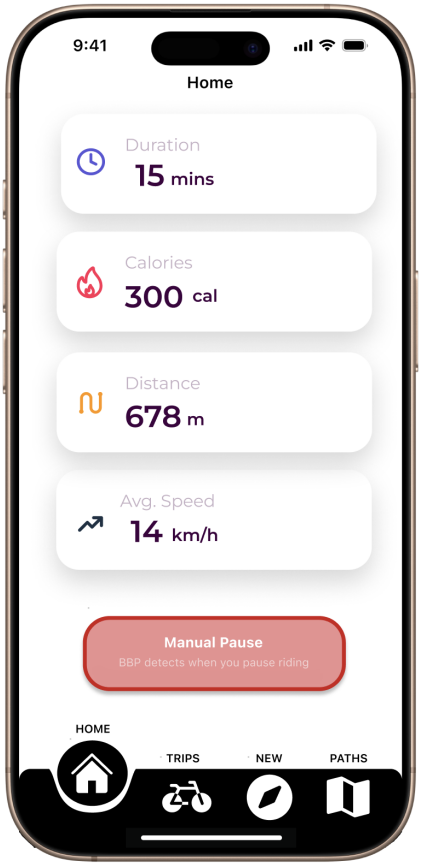
Live metrics appear only during an actual ride. As soon as a trip is initiated or activated manually, the central area is replaced seamlessly by the real-time metrics.

Navigation is provided by a persistent bottom menu that is always available. The menu contains three entries that correspond to the other core functions of the application: access to `Bike Paths`, access to `past trips`, and `Manual Path`.

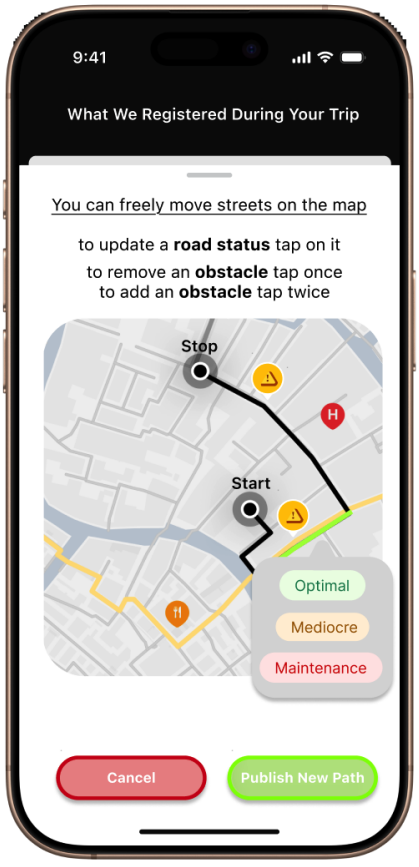
The user will be prompted when they end a trip in order to eventually update it. By interacting with the UI, they can perform certain actions, such as: change road status, add potholes or remove false positives readings about them. After finishing the update, the user can decide to publish it only if the bike path is considered safe; if a road is considered unsafe it should be marked on the UI with an error message.



(a) Home screen at start up



(b) Home screen during a trip



(c) Revising a path

Past trip recordings

In this section the user will be able to see a list of all their past recorded trips ordered by date with general metrics. Each recorded trip item can be clickable in the final product, this will take the user to a dedicated page where the trip selected is explained in details with options to publish it after reviewing.
As per the auto generated paths, if a trip is composed by an invalid road, it should be marked and therefore modified by the user to publish the trip as a bike path.

In this screen a *"profile"* button is also present; this is done in order to give access to the user stored personal information, modify it or delete the profile.

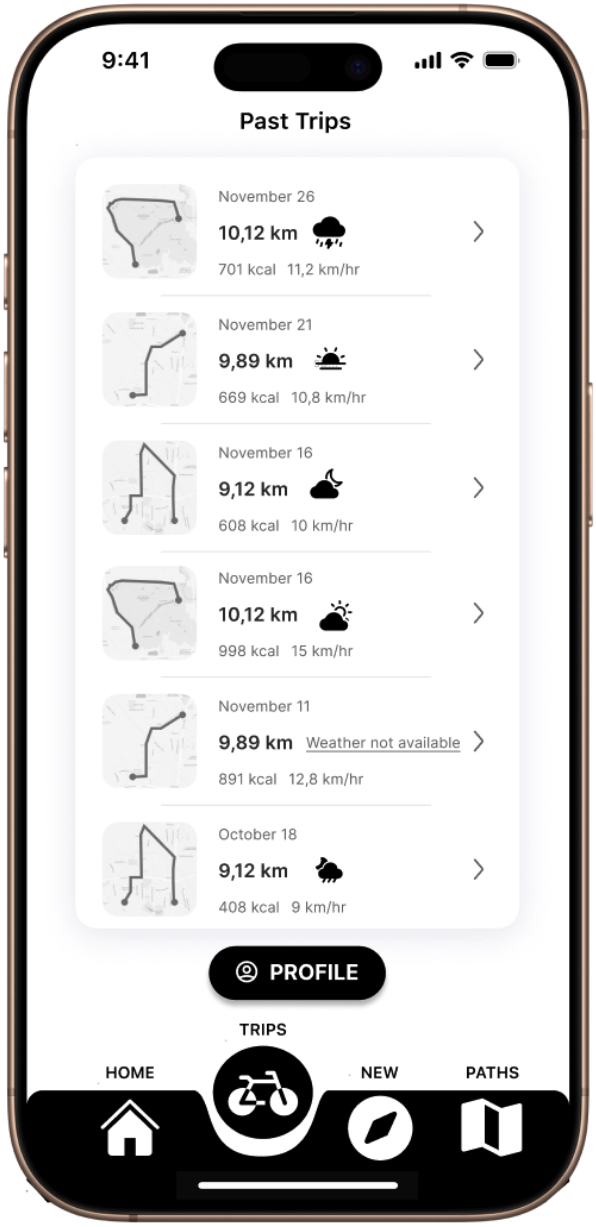


Figure 3.3: Past trip list

Manual path insertion

In this screen the user will be able to add, one by one, the streets composing the path they want to publish, alongside with the road status. The road status is asked when adding a new stop (the pop up menu is not shown in this document) As for the auto-generated trip, the user can also add potholes and obstacles by simply interacting with the UI.

The user is inhibited to add certain road, not compliant with the platform policy.

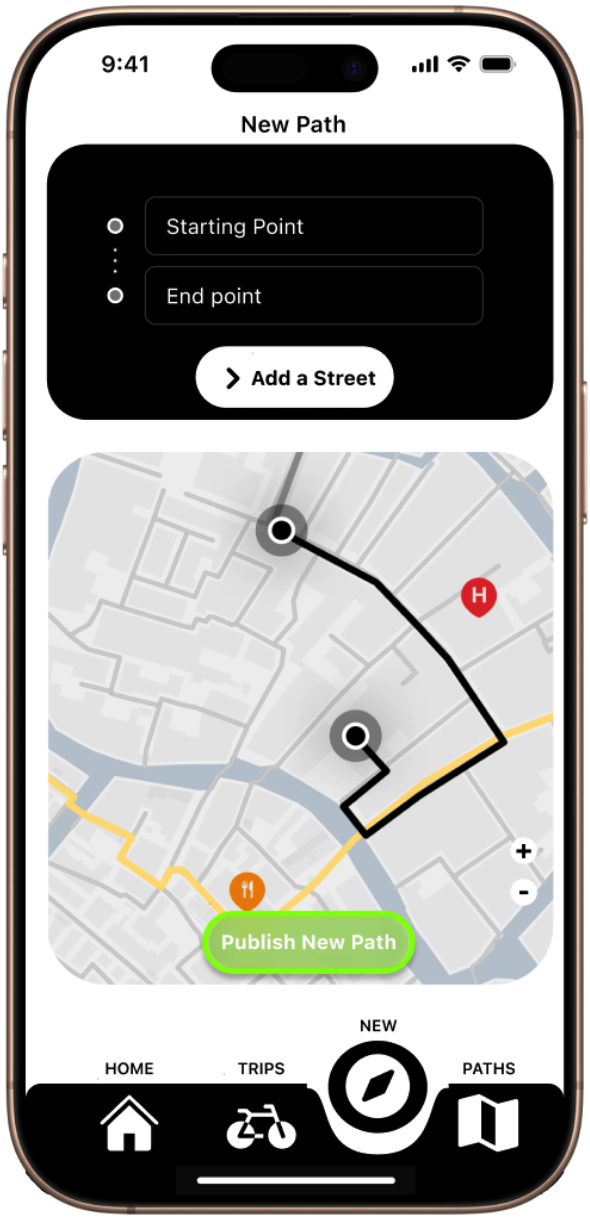


Figure 3.4: Inserting a new Path manually

Bike path search & results

In this screen the user will be able to insert origin and destination and see the relevant bike path for their research, alongside their own overall score and ordered following the specification. The user can scroll through all possible result and by clicking on a path card they will be taken to an in-depth view of selected path.

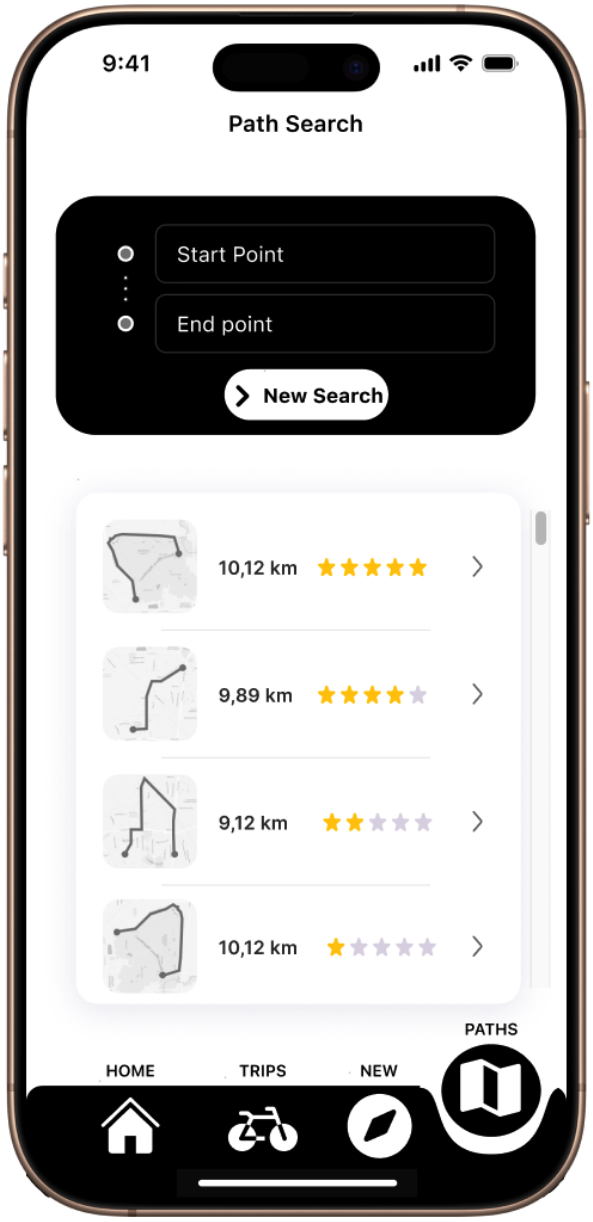


Figure 3.5: Searching new paths

3.1.2. Hardware Interfaces

BBP targets Android/iOS smartphones, because people carry them during rides. Phones also expose a rich set of on-board sensors and are able to compute and render the app itself, so they're ideal for continuous capture while cycling.
BBP relies on:

- **GPS Interface**
  Provides positioning and speed for trip recording and path tracing. Typical rate: ∼1 Hz during recording.
- **Accelerometer & Gyroscope**
  Raw motion samples to detect potholes/irregularities and support auto pause.
- **Display / UI**
  Renders HUD metrics, maps, path review and lists.
- **Haptics / Vibration**
  Tactile alerts for start/stop and obstacle warnings.
- **Local Storage**
  On-device persistence for trips, cache of paths and weather.
- **Network Interface (Wi-Fi / Cellular)**
  Connectivity for path search, weather, and profile sync.

3.1.3. Software Interfaces

The system interacts with many interfaces, categorized as internal services and external APIs, to provide its features.

- Internal Services
  - **Authentication Service API** provides user authentication and authorization. When a user successfully login, the service generates a token that is used to authenticate all future request.
  - **Notification Service API** handles the delivery of notifications to users through dedicated channels (e.g., email), in order to inform users about events.
  - **Database Service API** allows the system to save and retrieve data from the platform's DBMS (database management system).
- External APIs
  - **Weather-forecast Service API** is used to retrieve weather data and integrate it into the user's trips.
  - **Navigation-system Service API** is used for evaluating street compliance, defining what kind of roads the user can include in the making of bike paths.

3.1.4. Communication interfaces

- **HTTPS**
  Primary channel between the mobile app and backend Service APIs (REST/JSON).
- **OAuth**
  Authentication and authorization.
- **External Service APIs over HTTPS**
  Weather and Safety/Navigation providers invoked by the backend via HTTPS. Timeouts, with limited retries, and result caching are enforced.

3.2. Functional Requirements

- **Account Management**
  - **[R1]** The system shall allow potential service consumers to create accounts.
  - **[R2]** Users shall be able to securely login and logout using a username-password system or third-party authentication.
  - **[R3]** Users shall be able to recover their accounts using a password-recovery feature.
  - **[R4]** The system shall present a consent flow to collect explicit permission for background location/activity recognition and sensor data processing, storing such decision.
  - **[R5]** Registered users shall be able to configure preferred units of measure and language.
- **Registered and Unregistered Users Features**
  - **[R6]** The system shall accept user input for origin and destination and display all possible bike paths, ordered by score.
- **Registered Users Features**
  - **[R7]** Registered users shall be able to manually create bike paths (i.e.,"Manual Mode") by providing: a starting point; a finishing point; an ordered sequence of streets, specifying their overall status and the presence of irregularities (e.g., potholes).
  - **[R8]** Registered users shall be able to evaluate, modify (if necessary) and approve potential obstacles detected by the system in Automatic Mode for a bike path, assessing the correctness of information before publication.
  - **[R9]** Registered users shall be able to delete their profiles.
  - **[R10]** Registered users shall be able to monitor trip metrics.
  - **[R11]** Registered users shall be able to record a trip.
- **Trip Metrics System**
  - **[R12]** The system shall record personal trips and present statistics (distance, time, average speed) and, when available, weather data associated with the trip period.

  - **[R13]** The system shall allow offline trip recording (street-queuing data et al) and automatically sync when connectivity is restored.

- **Automatic Bike Path Generator**
  - **[R14]** The system shall be able to automatically create bike paths (i.e., "Automatic Mode") by processing data from the sensors of the device of the user (GPS, accelerometer, gyroscope).
  - **[R15]** The system shall be able to save manually created bike paths, allowing registered users to create bike paths in "Manual Mode".
  - **[R16]** The system shall support auto-pause/auto-resume when the user stops/moves again below/above thresholds.
  - **[R17]** The system shall verify the compliance of streets, allowing streets that satisfy defined criteria to be included in bike paths (e.g., average bike speed VS road limit speed, roadway width, traffic volume, dedicated bike lanes).
  - **[R18]** The system shall prevent registered users from publishing bike paths with unallowed streets.
- **Bike Path Assembler**
  - **[R19]** The system shall merge multiple contributions related to the same streets using at least freshness and number of confirmations, propagating the update to bike paths that include them.

3.2.1. Requirements Mapping

This section presents the correspondence between the system Goals (G) with the Functional Requirements (R) and the Domain Assumptions and Dependencies (D) that, together, allow for the goals themselves. Formally:

$$R \wedge D \models G.$$

For each goal, the specific requirements define the system behavior needed to achieve it, under the assumptions that describe the external conditions of the environment.

Goal 1: Enable the cycling community to access a shared catalog of safe and reliable bike paths.

Requirements	Assumptions and Dependencies
R6: The system shall accept user input for origin and destination and display all possible bike paths, ordered by score.	A1: Compliance with guidelines: All users (registered and unregistered) are expected to follow the platform's rules and guidelines and provide accurate, up-to-date information.
R7: Registered users shall be able to manually create bike paths (i.e.,"Manual Mode") by providing: a starting point; a finishing point; an ordered sequence of streets, specifying their overall status and the presence of irregularities (e.g., potholes).	A3: User's mobile device: The user must possess a mobile device (e.g., smartphone) with integrated sensors (mandatory: GPS, accelerometer, gyroscope) capable of internet communication and application processing.
R14: The system shall be able to automatically create bike paths (i.e., "Automatic Mode") by processing data from the sensors of the device of the user (GPS, accelerometer, gyroscope).	A4: Sensor reliability: GPS, accelerometer, gyroscope, and other necessary sensors are available, functioning correctly, and calibrated on the user's device.
R15: The system shall be able to save manually created bike paths, allowing registered users to create bike paths in "Manual Mode".	A5: GPS data detection: The error margin of the GPS signal shall be within acceptable thresholds to allow for the detection of the starting point (origin) and arrival point (destination).
R17: The system shall verify the compliance of streets, allowing streets that satisfy defined criteria to be included in bike paths (e.g., average bike speed VS road limit speed, roadway width, traffic volume, dedicated bike lanes).	A6: Accelerometer and Gyroscope sensitivity: Data detection from accelerometer and gyroscope sensors must be accurate enough to correctly detect potholes and other street irregularities.
R18: The system shall prevent registered users from publishing bike paths with unallowed streets.	D1: Reliable internet access: All users (registered and unregistered) shall have a stable internet connection to perform certain actions.

R19: The system shall merge multiple contributions related to the same streets using at least freshness and number of confirmations, propagating the update to bike paths that include them.	D3: Navigation-system Service API: The navigation system service API must be correctly implemented, sufficiently available, and functioning correctly.

Table 3.1: Goal 1 mapping table

Goal 2: Allow registered users to store trips with automatically-collected sensor data and associated weather (if available).

Requirements	Assumptions and Dependencies
R10: Registered users shall be able to monitor trip metrics.	A3: User's mobile device: The user must possess a mobile device (e.g., smartphone) with integrated sensors (mandatory: GPS, accelerometer, gyroscope) capable of internet communication and application processing.
R11: Registered users shall be able to record a trip.	A4: Sensor reliability: GPS, accelerometer, gyroscope, and other necessary sensors are available, functioning correctly, and calibrated on the user's device.
R12: The system shall record personal trips and present statistics (distance, time, average speed) and, when available, weather data associated with the trip period.	A5: GPS data detection: The error margin of the GPS signal shall be within acceptable thresholds to allow for the detection of the starting point (origin) and arrival point (destination).
R13: The system shall allow offline trip recording (street-queuing data et al) and automatically sync when connectivity is restored.	A6: Accelerometer and Gyroscope sensitivity: Data detection from accelerometer and gyroscope sensors must be accurate enough to correctly detect potholes and other street irregularities.
R16: The system shall support auto-pause/auto-resume when the user stops/moves again below/above thresholds.	D1: Reliable internet access: All users (registered and unregistered) shall have a stable internet connection to perform certain actions.

	D2: Weather-forecast Service API: The weather forecast service API must be correctly implemented, sufficiently available, and functioning correctly.

Table 3.2: Goal 2 mapping table

Goal 3: Develop and maintain a collaborative inventory of bike paths and their score, along with status and obstacle information regarding the streets they are composed of.

Requirements	Assumptions and Dependencies
R7: Registered users shall be able to manually create bike paths (i.e.,"Manual Mode") by providing: a starting point; a finishing point; an ordered sequence of streets, specifying their overall status and the presence of irregularities (e.g., potholes).	A1: Compliance with guidelines: All users (registered and unregistered) are expected to follow the platform's rules and guidelines and provide accurate, up-to-date information.
R8: Registered users shall be able to evaluate, modify (if necessary) and approve potential obstacles detected by the system in Automatic Mode for a bike path, assessing the correctness of information before publication.	A3: User's mobile device: The user must possess a mobile device (e.g., smartphone) with integrated sensors (mandatory: GPS, accelerometer, gyroscope) capable of internet communication and application processing.
R14: The system shall be able to automatically create bike paths (i.e., "Automatic Mode") by processing data from the sensors of the device of the user (GPS, accelerometer, gyroscope).	A4: Sensor reliability: GPS, accelerometer, gyroscope, and other necessary sensors are available, functioning correctly, and calibrated on the user's device.
R17: The system shall verify the compliance of streets, allowing streets that satisfy defined criteria to be included in bike paths (e.g., average bike speed VS road limit speed, roadway width, traffic volume, dedicated bike lanes).	A5: GPS data detection: The error margin of the GPS signal shall be within acceptable thresholds to allow for the detection of the starting point (origin) and arrival point (destination).

R19: The system shall merge multiple contributions related to the same streets using at least freshness and number of confirmations, propagating the update to bike paths that include them.	A6: Accelerometer and Gyroscope sensitivity: Data detection from accelerometer and gyroscope sensors must be accurate enough to correctly detect potholes and other street irregularities. D1: Reliable internet access: All users (registered and unregistered) shall have a stable internet connection to perform certain actions. D3: Navigation-system Service API: The navigation system service API must be correctly implemented, sufficiently available, and functioning correctly.

Table 3.3: Goal 3 mapping table

Goal 4: Enable registered users to contribute to the enrichment of the bike paths catalog, by publishing automatically-recorded bike paths.

Requirements	Assumptions and Dependencies
R8: Registered users shall be able to evaluate, modify (if necessary) and approve potential obstacles detected by the system in Automatic Mode for a bike path, assessing the correctness of information before publication. R14: The system shall be able to automatically create bike paths (i.e., "Automatic Mode") by processing data from the sensors of the device of the user (GPS, accelerometer, gyroscope).	A3: User's mobile device: The user must possess a mobile device (e.g., smartphone) with integrated sensors (mandatory: GPS, accelerometer, gyroscope) capable of internet communication and application processing. A4: Sensor reliability: GPS, accelerometer, gyroscope, and other necessary sensors are available, functioning correctly, and calibrated on the user's device.

R17: The system shall verify the compliance of streets, allowing streets that satisfy defined criteria to be included in bike paths (e.g., average bike speed VS road limit speed, roadway width, traffic volume, dedicated bike lanes). R18: The system shall prevent registered users from publishing bike paths with unallowed streets.	A5: GPS data detection: The error margin of the GPS signal shall be within acceptable thresholds to allow for the detection of the starting point (origin) and arrival point (destination). A6: Accelerometer and Gyroscope sensitivity: Data detection from accelerometer and gyroscope sensors must be accurate enough to correctly detect potholes and other street irregularities. D1: Reliable internet access: All users (registered and unregistered) shall have a stable internet connection to perform certain actions. D3: Navigation-system Service API: The navigation system service API must be correctly implemented, sufficiently available, and functioning correctly.

Table 3.4: Goal 4 mapping table

Goal 5: Enable registered users to contribute to the enrichment of the bike paths catalog, by publishing manually-entered bike paths.

Requirements	Assumptions and Dependencies
R7: Registered users shall be able to manually create bike paths (i.e.,"Manual Mode") by providing: a starting point; a finishing point; an ordered sequence of streets, specifying their overall status and the presence of irregularities (e.g., potholes).	A1: Compliance with guidelines: All users (registered and unregistered) are expected to follow the platform's rules and guidelines and provide accurate, up-to-date information.

R15: The system shall be able to save manually created bike paths, allowing registered users to create bike paths in "Manual Mode". R17: The system shall verify the compliance of streets, allowing streets that satisfy defined criteria to be included in bike paths (e.g., average bike speed VS road limit speed, roadway width, traffic volume, dedicated bike lanes). R18: The system shall prevent registered users from publishing bike paths with unallowed streets.	D1: Reliable internet access: All users (registered and unregistered) shall have a stable internet connection to perform certain actions. D3: Navigation-system Service API: The navigation system service API must be correctly implemented, sufficiently available, and functioning correctly.

Table 3.5: Goal 5 mapping table

Goal 6: Allow any user (registered or unregistered) to search for bike paths from Origin to Destination, ordered by their effectiveness for such parameters.

Requirements	Assumptions and Dependencies
R6: The system shall accept user input for origin and destination and display all possible bike paths, ordered by score. R19: The system shall merge multiple contributions related to the same streets using at least freshness and number of confirmations, propagating the update to bike paths that include them.	D1: Reliable internet access: All users (registered and unregistered) shall have a stable internet connection to perform certain actions.

Table 3.6: Goal 6 mapping table

Goal 7: Ensure bike paths consist only of roads suitable and safe for cycling.

Requirements	Assumptions and Dependencies

R17: The system shall verify the compliance of streets, allowing streets that satisfy defined criteria to be included in bike paths (e.g., average bike speed VS road limit speed, roadway width, traffic volume, dedicated bike lanes). R18: The system shall prevent registered users from publishing bike paths with unallowed streets.	D1: Reliable internet access: All users (registered and unregistered) shall have a stable internet connection to perform certain actions. D3: Navigation-system Service API: The navigation system service API must be correctly implemented, sufficiently available, and functioning correctly.

Table 3.7: Goal 7 mapping table

Goal 8: Reduce errors and false positives through user confirmation and information merging.

Requirements	Assumptions and Dependencies
R8: Registered users shall be able to evaluate, modify (if necessary) and approve potential obstacles detected by the system in Automatic Mode for a bike path, assessing the correctness of information before publication. R19: The system shall merge multiple contributions related to the same streets using at least freshness and number of confirmations, propagating the update to bike paths that include them.	A4: Sensor reliability: GPS, accelerometer, gyroscope, and other necessary sensors are available, functioning correctly, and calibrated on the user's device. A5: GPS data detection: The error margin of the GPS signal shall be within acceptable thresholds to allow for the detection of the starting point (origin) and arrival point (destination). A6: Accelerometer and Gyroscope sensitivity: Data detection from accelerometer and gyroscope sensors must be accurate enough to correctly detect potholes and other street irregularities.

	D1: Reliable internet access: All users (registered and unregistered) shall have a stable internet connection to perform certain actions.

Table 3.8: Goal 8 mapping table

Goal 9: Support a secure account lifecycle for registered users, including authentication, recovery, consent capture, and preferences management.

Requirements	Assumptions
R1: The system shall allow potential service consumers to create accounts. R2: Users shall be able to securely login and logout using a username-password system or third-party authentication. R3: Users shall be able to recover their accounts using a password-recovery feature. R4: The system shall present a consent flow to collect explicit permission for background location/activity recognition and sensor data processing, storing such decision. R5: Registered users shall be able to configure preferred units of measure and language. R9: Registered users shall be able to delete their profiles.	A2: Valid credentials: To use the service, registered users are expected to have a valid email address. D1: Reliable internet access: All users (registered and unregistered) shall have a stable internet connection to perform certain actions.

Table 3.9: Goal 9 mapping table

Coverage Table: $R \wedge D \models G$.

	G1	G2	G3	G4	G5	G6	G7	G8	G9
R1									✓
R2									✓
R3									✓
R4									✓
R5									✓
R6	✓					✓			
R7	✓		✓		✓				
R8			✓	✓				✓	
R9									✓
R10		✓							
R11		✓							
R12		✓							
R13		✓							
R14	✓		✓	✓					
R15	✓				✓				
R16		✓							
R17	✓		✓	✓	✓		✓		
R18	✓			✓	✓		✓		
R19	✓		✓			✓		✓	
A1	✓		✓		✓				
A2									✓
A3	✓	✓	✓	✓					
A4	✓	✓	✓	✓				✓	
A5	✓	✓	✓	✓				✓	
A6	✓	✓	✓	✓				✓	
D1	✓	✓	✓	✓	✓	✓	✓	✓	✓
D2		✓							
D3	✓		✓	✓	✓		✓		

Table 3.10: Requirements Mapping Coverage Table

3.2.2. Use case diagram

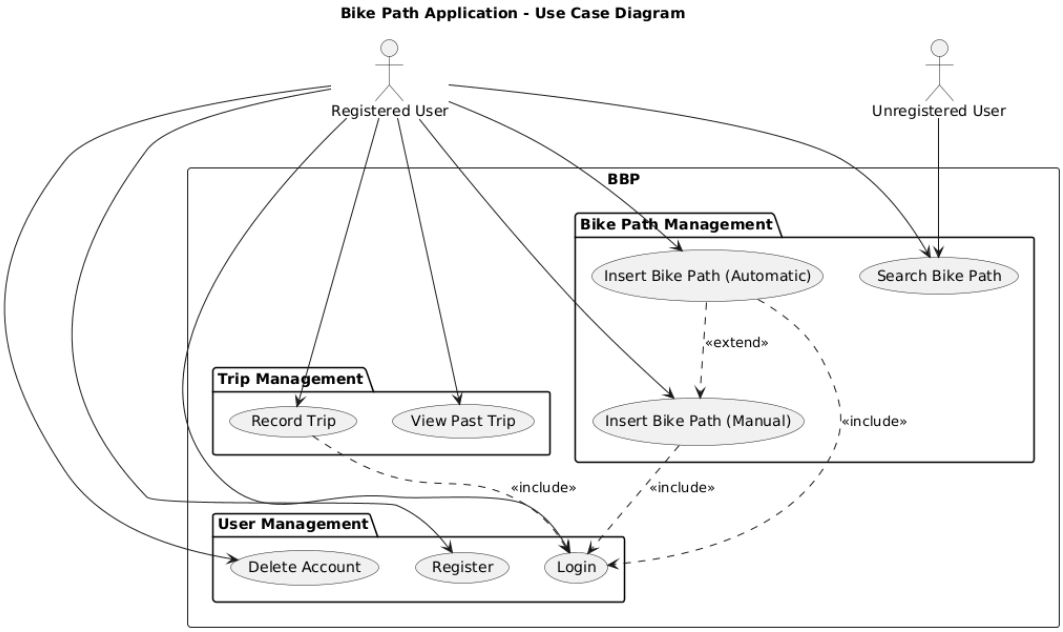


Figure 3.6: Use Case Diagram

3.2.3. Use Cases

[UC1] User registration

Name	User registration
Actors	Unregistered users
Entry Condition	The user has opened the platform (BBP).
Event Flow	• (a) The user clicks the "Sign up" button. • (b) The system shows a registration form to the user. • (c) The user fills in the form. • (d) The user presses the "Register" button to confirm and send their data. • (e) The system confirms the registration and it redirects the user to the login page.
Exit Condition	The user has successfully registered and they can now log into the platform.
Exception	• (c) The system detects that the inserted email is already in use. • (d) The system detects that the registration form is missing some compulsory information or any invalid input.

Table 3.11: UC1 Table

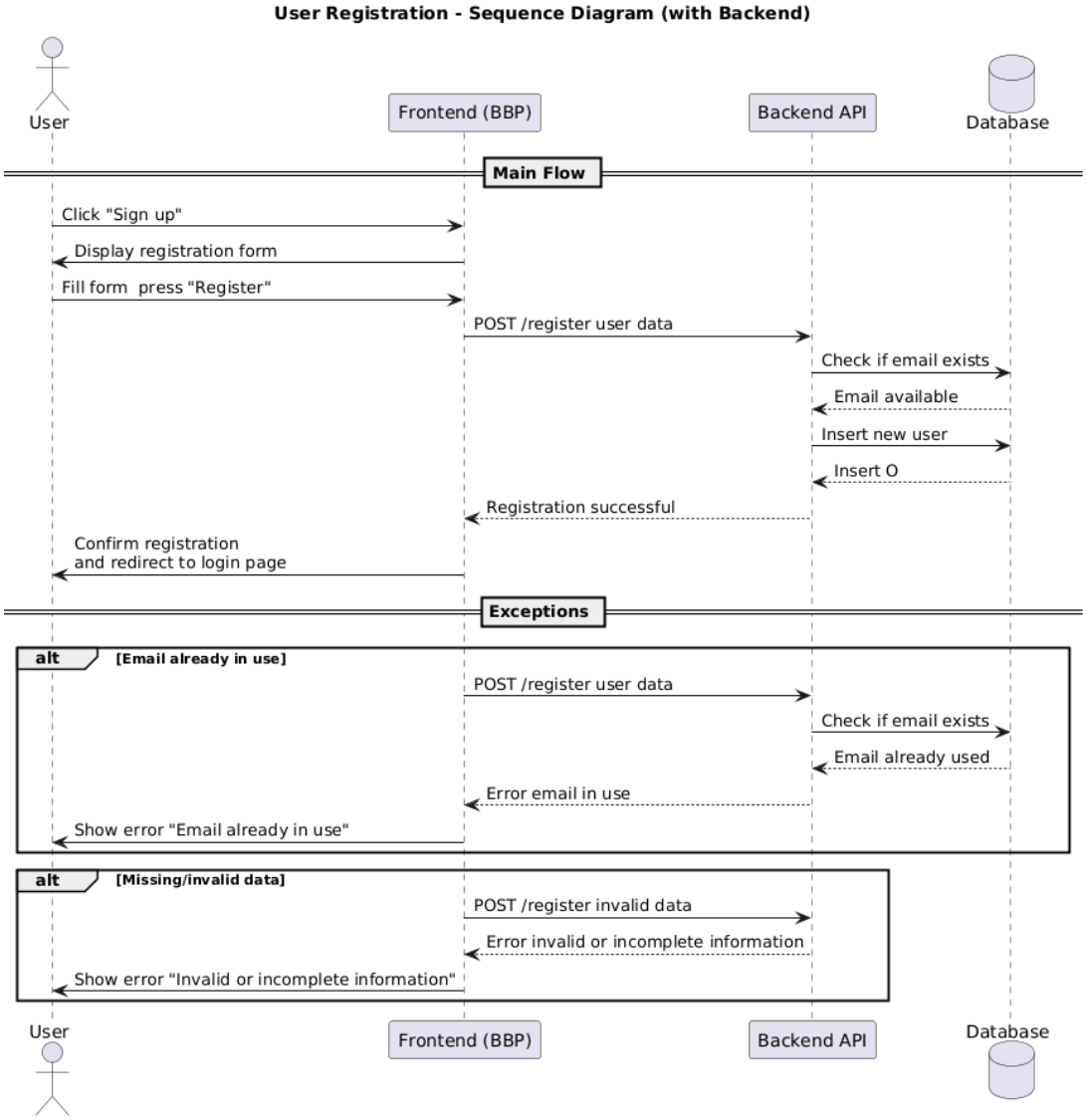


Figure 3.7: UC1 Sequence Diagram

[UC2] Registered User login

Name	Registered User login
Actors	• Registered Users • Unregistered Users
Entry Condition	The user opens the application (BBP).
Event Flow	• (a) The system prompts the user to insert their email/username and password in a login form. Alternatively, the system offers an external authentication. • (b) The user fills in the form with the required data or they log in with the third party SSO method. • (c) The system verifies the credentials against the database. • (d) If credentials are correct, the system redirects the user to the home page. • (e) If credentials are wrong, the system prompts the user to try again or to recover their credentials.
Exit Condition	The user is logged in successfully into the platform.
Exception	(c) The system detects invalid credentials.

Table 3.12: UC2 Table

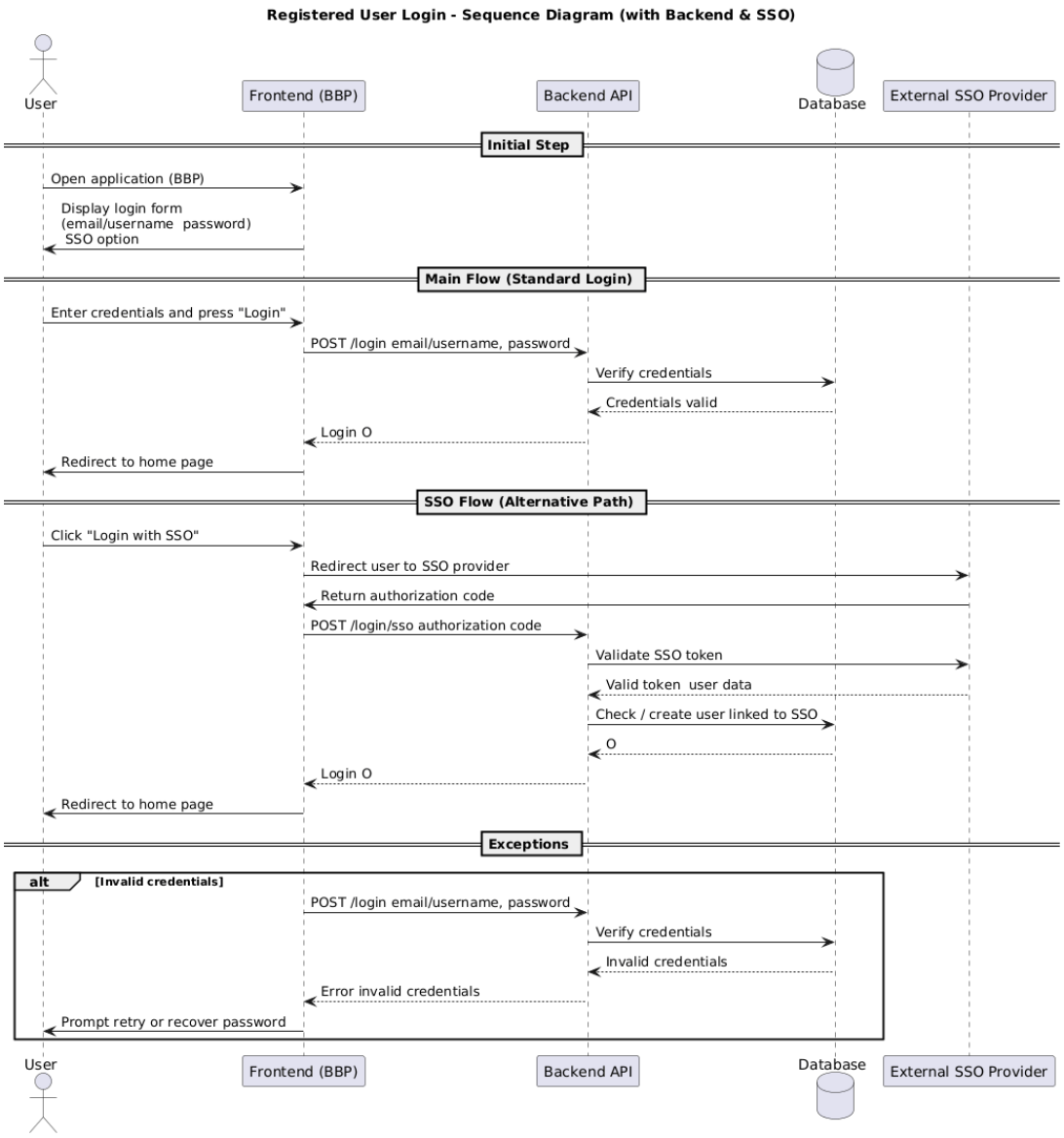


Figure 3.8: UC2 Sequence Diagram

[UC3] Registered User deletion

Name	Registered User deletion
Actors	Registered Users
Entry Condition	The user is already logged in the application and they are on the settings.
Event Flow	• (a) The user clicks on "Delete your account". • (b) The system asks the user if they are sure of the action. • (c) The user confirms their intentions. • (d) The systems deletes all the information regarding the user. • (e) The system redirects the user to the login page.
Exit Condition	The account is deleted from backend.
Exception	

Table 3.13: UC3 Table

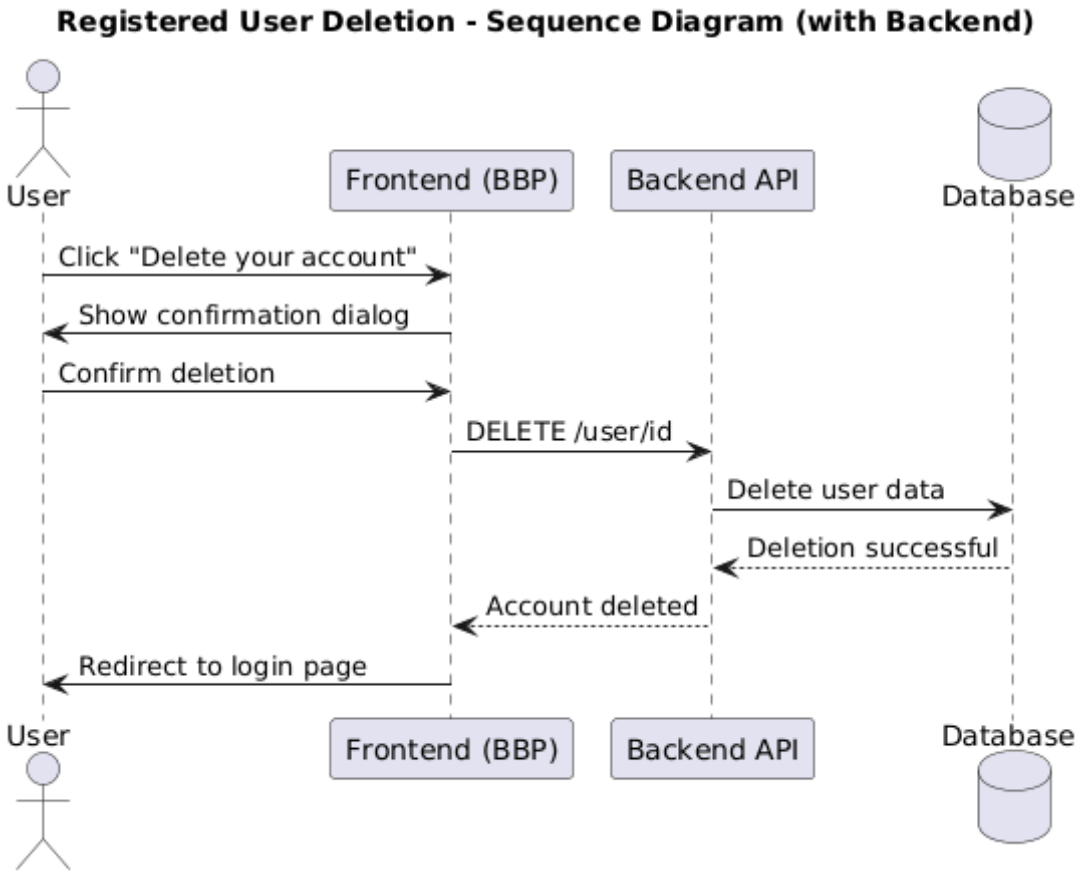


Figure 3.9: UC3 Sequence Diagram

[UC4] Registered User records a Trip

Name	Registered User records a Trip
Actors	Registered users
Entry Condition	The user is on the home page of the application (BBP).
Event Flow	• (a) The user presses on "Start Recording" or starts cycling. • (b) The system detects that cycling session has started and begins with collecting GPS and IMU data from the sensors. • (c) The user stops manually the recording of the trip or stops cycling. • (d) The system detects that trip has been stopped. • (e) The system calls the Weather APIs, providing temporal and spatial data of the recorded trip. • (f) The APIs reply with the required data. • (g) The system collects the weather data. • (h) The system computes the performance metrics of the trip. • (i) The system shows to the user the summary of the trip.
Exit Condition	The user stops the recording of the trip.
Exception	(f) Weather is not available, APIs don't reply.

Table 3.14: UC4 Table

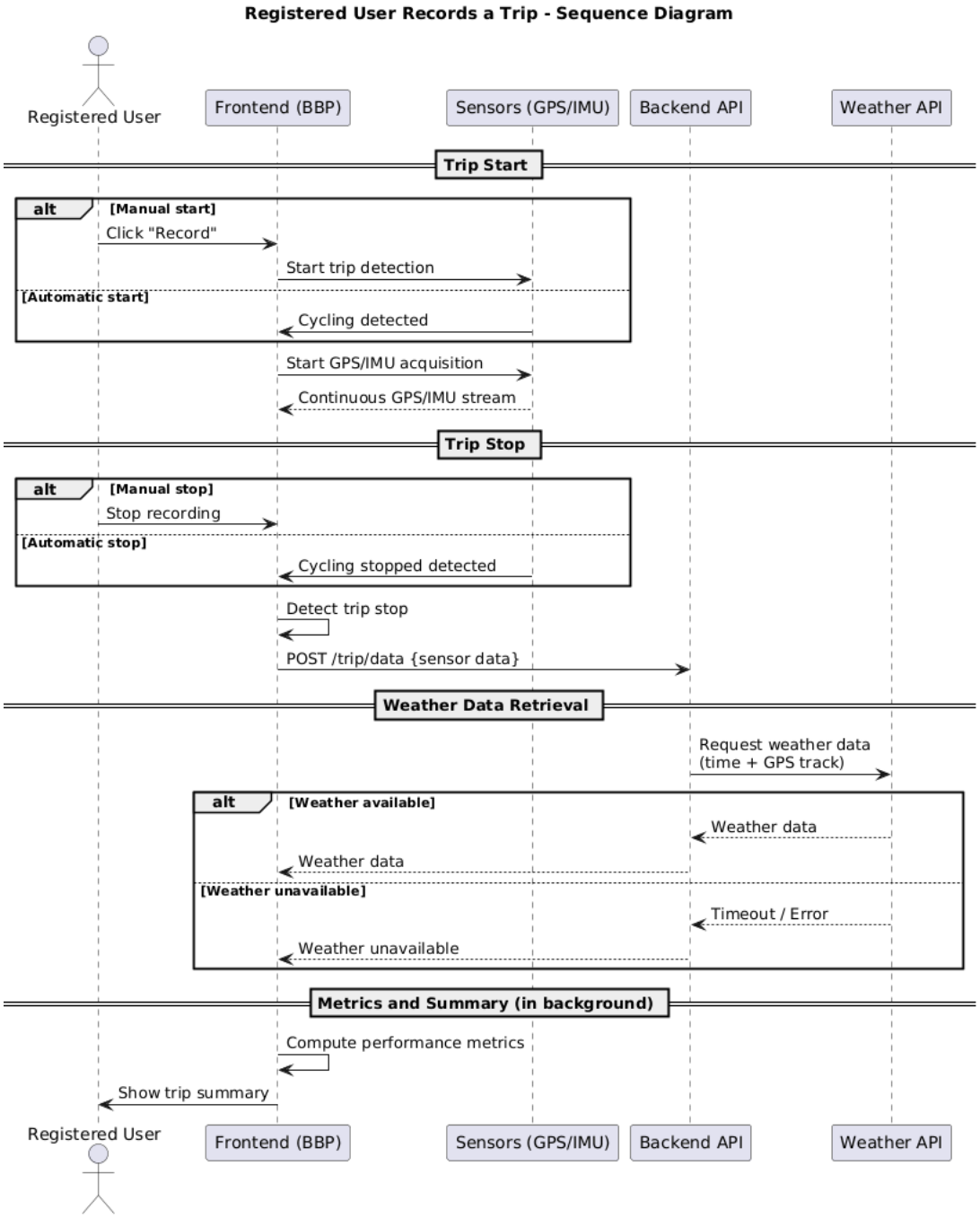


Figure 3.10: UC4 Sequence Diagram

[UC5] Registered User sees a past Trip

Name	Registered User sees a past Trip
Actors	Registered Users
Entry Condition	The user is in the home page.
Event Flow	• (a) The user clicks on the "Trips" tab. • (b) The system retrieves from the database all user's past trips. • (c) The system shows the collection of trips to the user. • (d) The user picks a past trip. • (e) The system retrieves from the database all the information about the trip. • (f) The system shows the retrieved data to the user.
Exit Condition	The user exits from the "Trips" tab.
Exception	(b) No past trips from that user exist on the database.

Table 3.15: UC5 Table

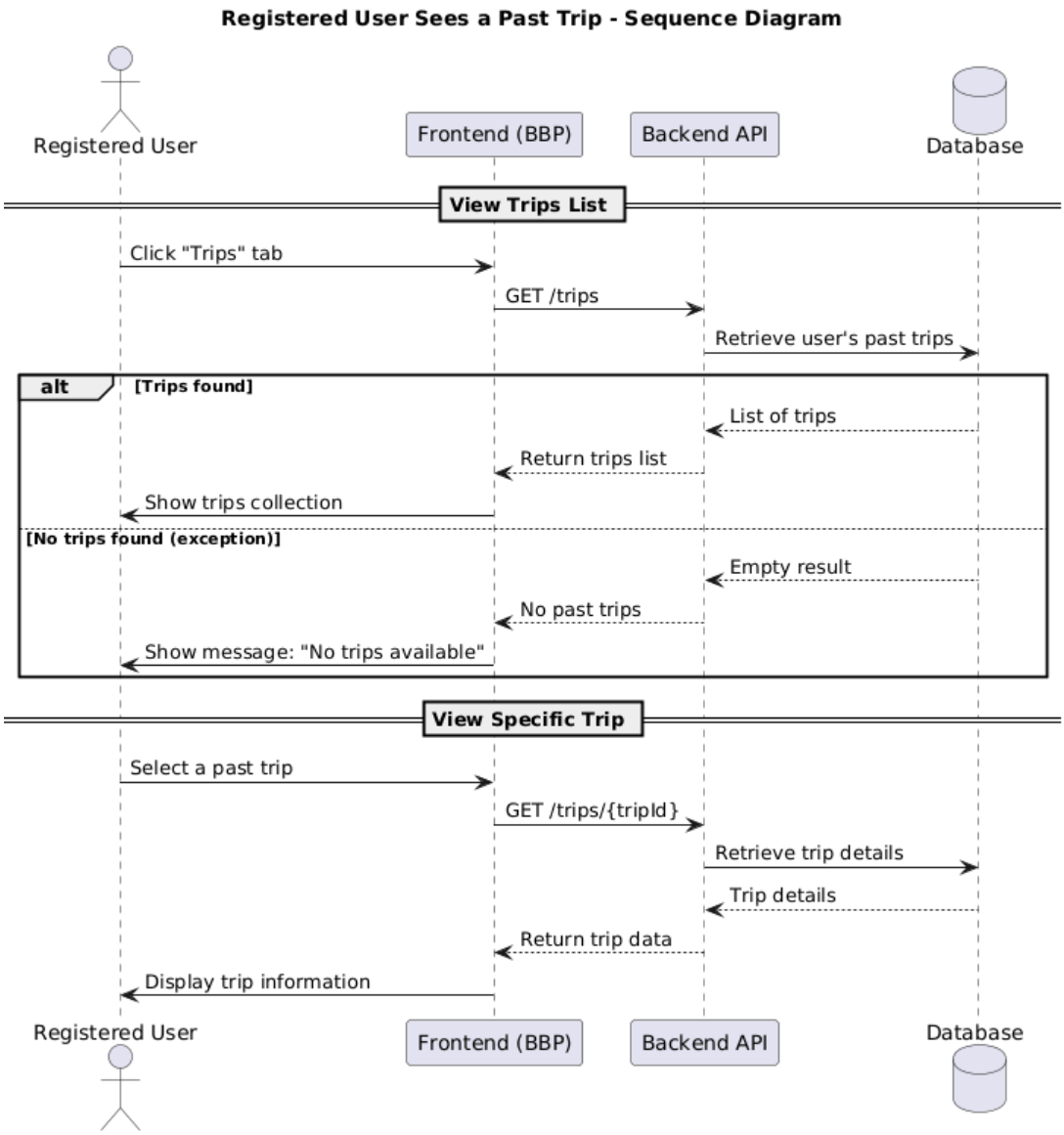


Figure 3.11: UC5 Sequence Diagram

[UC6] Registered User inserts a Bike Path in Manual Mode

Name	Registered User inserts a Bike Path in Manual Mode
Actors	Registered Users
Entry Condition	The user clicks on the "Add Bike Path".
Event Flow	• (a) The user provides starting point, ending point, the name of the streets (sequentially ordered), their status and eventual irregularities. • (b) The system calls the Navigation APIs to check if the inserted streets are suitable to be safe Bike Paths. • (c) If all streets are suitable, the Bike Path gets published. • (d) If at least one street is not suitable, the system prompts the user to edit the information to make it suitable.
Exit Condition	The Bike Path gets published or gets rejected.
Exception	(a) Invalid information is sent to the system.

Table 3.16: UC6 Table

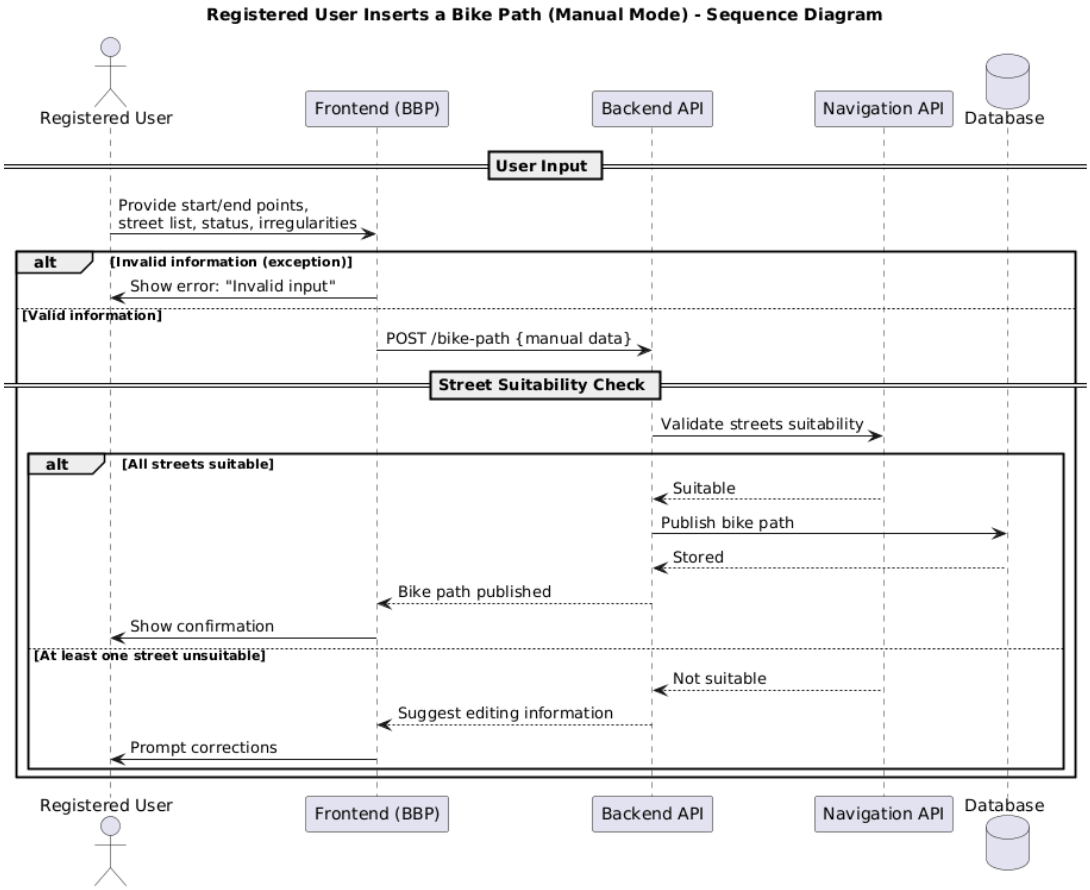


Figure 3.12: UC6 Sequence Diagram

[UC7] Registered User inserts a Bike Path in Automatic Mode

Name	Registered User inserts a Bike Path in Automatic Mode
Actors	Registered Users
Entry Condition	The user is looking at one personal trip and clicks the "publish" button.
Event Flow	• (a) The system processes the GPS and IMU data regarding a trip to detect streets and possible irregularities, and it shows them to the user. • (b) The system shows to the user the retrieved data and prompts them to review it. • (c) The user reviews the detected irregularities and they enter the status of the detected streets. • (e) If any corrections are required, the system fixes the automatically detected data. • (f) The system calls the Navigation APIs to verify suitability of the streets. • (g) If streets are suitable, the Bike Path gets published. • (h) If streets are not suitable, the Bike Path cannot be published.
Exit Condition	The Bike Path is published or rejected.
Exception	

Table 3.17: UC7 Table

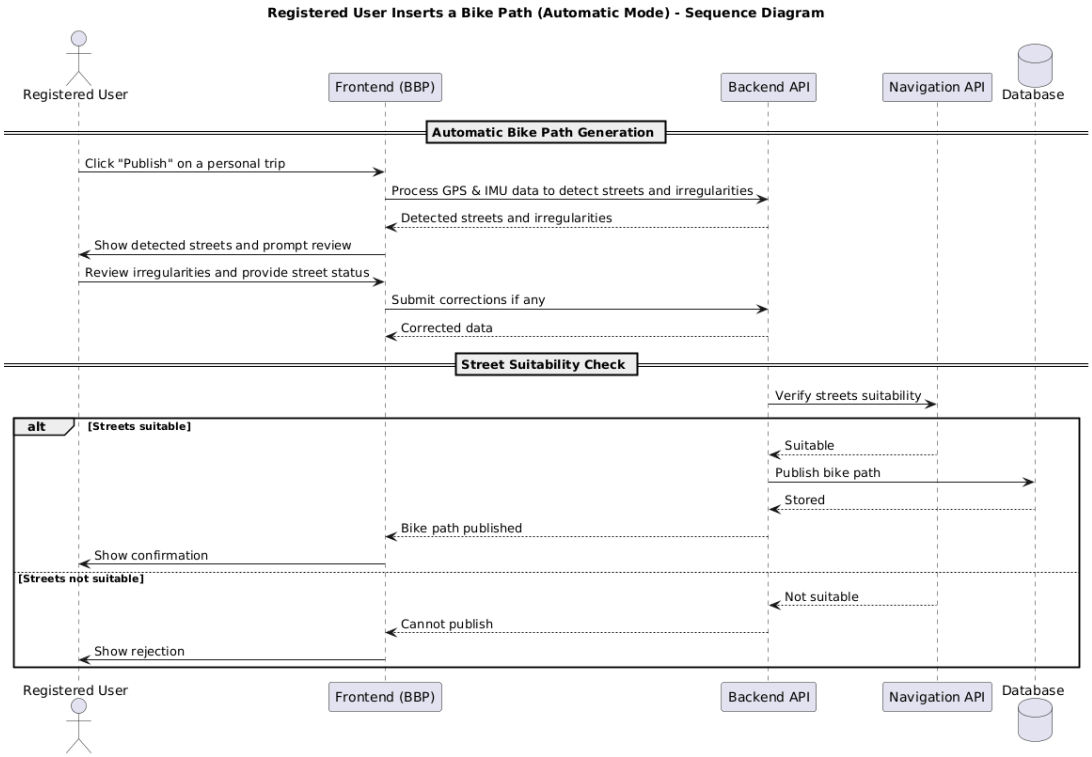


Figure 3.13: UC7 Sequence Diagram

[UC8] User searches a Bike Path

Name	User searches a Bike Path
Actors	• Registered Users • Unregistered Users
Entry Condition	The user clicks on the "Search" button.
Event Flow	• (a) The user specifies a Destination to reach with their bike starting from the Origin. • (b) The system retrieves all the Bike Paths that link the two points. • (c) The system computes the scores of the Bike Paths, according to their status and their efficiency to reach the destination. • (d) The system shows to the user the highest scoring Bike Paths, providing all the indications.
Exit Condition	The user picks a Bike Path or cancels the operation.
Exception	(b) No suitable Bike Paths are retrieved.

Table 3.18: UC8 Table

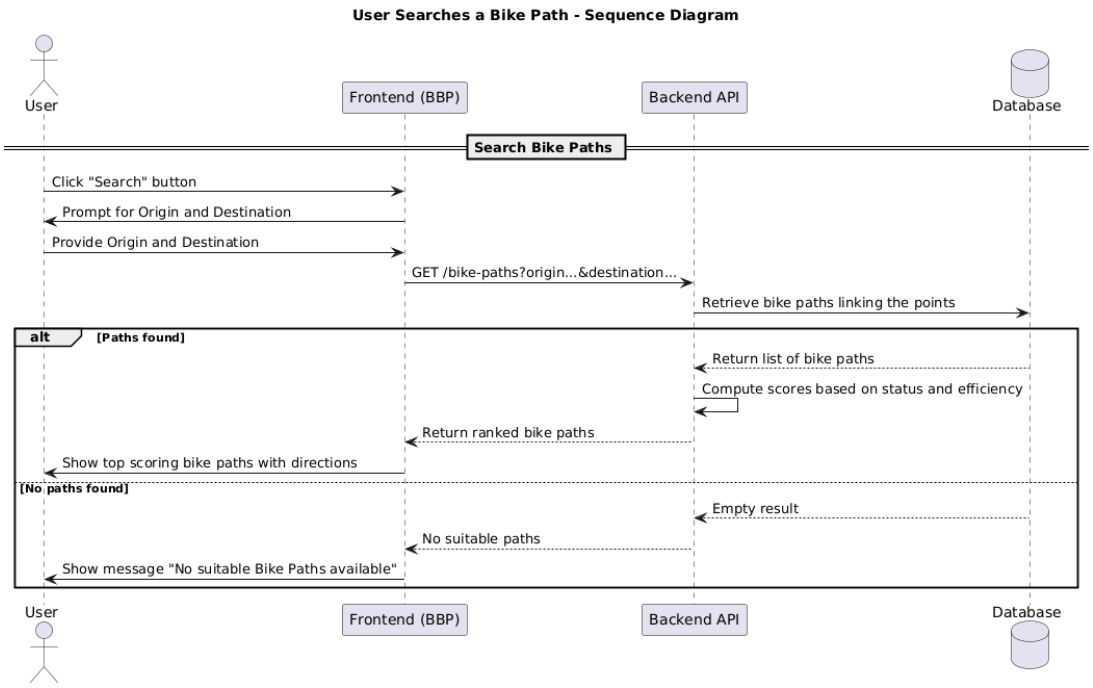


Figure 3.14: UC8 Sequence Diagram

3.3. Performance Requirements

3.3.1. Client

Responsiveness Targets

To ensure a usable experience, the app must:

- Acknowledge a user tap within **100 ms** (visual/haptic feedback).
- Complete the resulting UI action within **300 ms**.
- Delivery of push notifications (e.g., "path ready for review") within **30 s** of the triggering event.
- When fetching paths or other data from the backend it must **cache** this data with a reasonable TTL (around 30 minutes).

3.3.2. Server

In Italy there are approximately 22 million people who cycle at least once a week and about 3.8–4.0 million adults (between 18 and 69 years of age) ride daily for habitual trips. Assuming an initial participation of 5% of daily cyclists, we estimate around 200,000 daily users[1].

Even under burst conditions in which many users hit the backend simultaneously, the platform must guarantee:

- server responses to client data requests must be within **2 s**.
- service to at least **10%** of the daily average users as concurrent requests without perceptible degradation.

[1]Data retrieved from the "Istituto Superiore di Sanità" website

3.4. Design Constraints

3.4.1. Standards compliance:

- **Privacy and Data Protection Standards:**
  - The platform must comply with the european **General Data Protection Regulation (GDPR)** to protect the privacy and personal data of users and be able to operate in Europe.
- **Security Standards:**
  - Passwords and sensitive user data must be encrypted following **AES-256** encryption standards.
- **API and Interoperability Standards:**
  - The system APIs must facilitate integration with third-party systems by using standard protocols such as **HTTPS** and **JSON** authentication.

3.4.2. Hardware limitations

- the device must be able to connect to the internet communication.

BBP needs to run smoothly on mobile devices. Also, it needs to run in background execution while keeping the device usable and have an acceptable response time to user actions. Therefore the following requirements should be met:

- **Supported OS:** Android/iOS.
- **CPU:** with at least 4 cores; RAM $\geq$ 4 GB; sufficient free storage.
- **Hardware:** GPS, accelerometer and gyroscope.
- **Permissions**: background location and activity/motion recognition enabled.

3.4.3. Any other constraint

- Weather and Maps APIs should grant a good SLA according to the number of requests and system availability.

3.5. Software system attributes

3.5.1. Reliability

Since the app execution depends strictly on user Hardware performance, we focus our attention to what we can effectively control: **the backend infrastructure**.

To ensure reliability, the backend must guarantee a good fault tolerance in order to avoid service unavailability for login, path research and publication, for this purpose the system must also implement secure DB backups in order to guarantee the possibility to restore data even in presence of fatal failures.

3.5.2. Availability

The backend must have an uptime of at least 99.9% to ensure continuous availability. In order to guarantee these performances the backend may rely on a distributed system with high replication and consistency, avoiding single point of failures, indeed it would be recommended to use services (preferably) stateless, exposed behind a load balancer.

3.5.3. Security

- **User Data Protection:** All personal information, such as user data and personal trips, must be stored securely and only accessible to the authorized users.
- **Authentication and Authorization:** Users must log in using secure credentials, and the system should ensure that each user can only access their own data and public available data.
- **Secure Communications:** All personal user data exchanged with the backend must be encrypted to prevent interception.

3.5.4. Maintainability

- **Modular Architecture:** The BBP platform adopts a service-oriented (microservices) backend to enable independent updates or replacements of components without impacting the whole system.
- **Documentation:** All backend services and mobile app modules must be adequately documented (including APIs).
- **Automated Testing:** Critical paths must be covered by automated tests to prevent regressions during updates.

3.5.5. Portability

- **Portability (App):** The BBP app must run on Android and iOS and adapt to multiple screen size following Responsive Design paradigms.
- **Portability (Backend):** Backend services must be easily deployable on managed platforms. Configuration should allow cloud/provider migration with minimal effort.

4 | Formal Analysis using Alloy

This `Alloy` model formalizes the part of the BBP app that manages streets, reports, obstacles, and their aggregated status. It models entities such as `Street`, `Report`, and `Obstacle`, together with an enumeration for street conditions. In this way, we can automatically verify one of the core features of the app: each street's status remains consistent with the reports it receives, with particular attention to both the freshness of the data and the majority rule among conflicting reports.

```
1
2  open util/ordering[Time]  // total order on Time, used for freshness
3
4  // -------------------------------------------------------------
5  // SIGNATURES
6  // -------------------------------------------------------------
7
8  // generic user (cannot be instantiated directly)
9  abstract sig User {}
10
11 // registered user (can publish and create reports)
12 sig RegisteredUser extends User {}
13
14 // geographic point (start/end of BikePath)
15 sig GeoPoint {}
16
17 // time instant (totally ordered)
18 sig Time {}
19
20 // stret condition type, with associated penalty points
21 abstract sig Status { points: one Int }
22 one sig Optimal, Mediocre, NeedsMaintenance extends Status {}
23
```

```
24 // Obstacle type, each one with its own penalty
25 abstract sig ObstacleType { points: one Int }
26 one sig Pothole, Bump, GenericObstacle extends ObstacleType {}
27
28 sig Street {
29   status: one Status    // current, merged status of the street
30 }
31
32 // Bike path as formally defined in the RASD
33 sig BikePath {
34   start, end: one GeoPoint,
35   streets: some Street,
36   createdBy: one RegisteredUser,
37   score: one Int
38 }
39
40 // A standalone, recorded trip (always private) that can optionally be
   ↪ used to generate a BikePath (basePath)
41 sig Trip {
42   owner: one RegisteredUser,
43   from, to: one GeoPoint,
44   basePath: lone BikePath     // if present: BikePath obtained from this
     ↪ Trip
45 }
46
47 // A report created by a registered user while travelling a BikePath:
48 //   - relative to a specific BikePath and Street (max one Report per
   ↪ Street)
49 //   - timestamped with Time
50 //   - carries an observed Status and a (possibly empty) set of Obstacles
51 sig Report {
52   by: one RegisteredUser,
53   path: one BikePath,
54   street: one Street,
55   createdAt: one Time,
56   rStatus: one Status,
57   rObstacles: set ObstacleType
```

```
58 }
59
60 // ---------------------------------------------------------------
61 // BASIC FACTS FROM SPECIFICATIONS
62 // ---------------------------------------------------------------
63
64 // Higher points = worse, for the system
65 fact PointValues {
66   Optimal.points = 0
67   Mediocre.points  = 1
68   NeedsMaintenance.points = 2
69
70   Pothole.points = 3
71   Bump.points = 2
72   GenericObstacle.points = 1
73 }
74
75 // Report consistency: every report for a BikePath refers to a Street
   ↪  that actually belongs to that path
76 fact ReportsConsistentWithPaths {
77   all r: Report | r.street in r.path.streets
78 }
79
80 // Forbid circular BikePaths (start == end).
81 fact NoCircularPaths {
82   all bp: BikePath | bp.start != bp.end
83 }
84
85 // ---------------------------------------------------------------
86 // HELPER FUNCTIONS
87 // ---------------------------------------------------------------
88
89 // Get tutti i report di una certa Street (indipendentemente dalla
   ↪  BikePath)
90 fun reportsForStreet[s: Street]: set Report {
91   { r: Report | r.street = s }
92 }
```

```
93
94  // Get the most recent time among all reports for a street
95  fun mostRecentTime[s: Street]: set Time {
96    { t: reportsForStreet[s].createdAt |
97      no t2: reportsForStreet[s].createdAt | t in t2.^next
98    }
99  }
100
101 // Get all reports with maximal freshness (latest time, aka more recent)
    ↪  for a street
102 fun mostRecentReports[s: Street]: set Report {
103   { r: reportsForStreet[s] | r.createdAt in mostRecentTime[s] }
104 }
105
106 // Count number of reports with a specific status among a set of reports
107 fun countReportsWithStatus[rs: set Report, st: Status]: Int {
108   # { r: rs | r.rStatus = st }
109 }
110
111 / ----------------------------------------------------------------
112 // MERGING INFO FOR STREET (freshness + confirmations, RASD Req19)
113 // Core logic
114 // ----------------------------------------------------------------
115
116 // Select status for a street in a way that is majority-based and
    ↪  freshness-aware
117 //  1. if no reports, keep existing status (the default initial value)
118 //  2. if exactly one most-recent report, use such status
119 //  3. if several most-recent reports, choose the status with highest
    ↪  count
120 //       - in case of tie on (3), select among tied ones the one with
    ↪  minimal points ("better" status)
121
122 fun computedStatus[s: Street]: Status {
123   let rs = mostRecentReports[s] |
124     (no rs) => s.status
125     else (one rs) => (rs.rStatus)
```

```
126     else {
127       let optCnt = countReportsWithStatus[rs, Optimal] |
128       let medCnt = countReportsWithStatus[rs, Mediocre] |
129       let nmCnt  = countReportsWithStatus[rs, NeedsMaintenance] |
130
131       // strict majority cases
132       (optCnt > medCnt and optCnt > nmCnt) => Optimal
133       else (medCnt > optCnt and medCnt > nmCnt) => Mediocre
134       else (nmCnt > optCnt and nmCnt > medCnt) => NeedsMaintenance
135
136       // tie cases: choose among the tied statuses the one with best
          ↪ (lowest) points
137       else {
138         // tie Optimal-Mediocre
139         (optCnt = medCnt and optCnt > nmCnt) =>
140           (Optimal.points <= Mediocre.points) => Optimal else Mediocre
141
142         // tie Optimal-NeedsMaintenance
143         else (optCnt = nmCnt and optCnt > medCnt) =>
144           (Optimal.points <= NeedsMaintenance.points) => Optimal else
              ↪ NeedsMaintenance
145
146         // tie Mediocre-NeedsMaintenance
147         else (medCnt = nmCnt and medCnt > optCnt) =>
148           (Mediocre.points <= NeedsMaintenance.points) => Mediocre else
              ↪ NeedsMaintenance
149
150         // all equal (optCnt = medCnt = nmCnt): choose globally best
            ↪ status
151         else
152           (Optimal.points <= Mediocre.points and Optimal.points <=
              ↪ NeedsMaintenance.points)
153             => Optimal
154           else (Mediocre.points <= NeedsMaintenance.points)
155             => Mediocre
156           else
157             NeedsMaintenance
```

```
158       }
159     }
160 }
161
162 // This models the "update-after-merge" behavior: street status must
    ↪  reflect the computed status from reports
163 fact StatusUpdatedByReports {
164   all s: Street |
165     some reportsForStreet[s] implies s.status = computedStatus[s]
166 }
167
168 / ------------------------------------------------------------
169 // ASSERTIONS
170 // ------------------------------------------------------------
171
172 // Score is always non-negative (since all points are non-negative)
173 assert NonNegativeScore {
174   all bp: BikePath | bp.score >= 0
175 }
176
177 // All BikePaths are made only of Street elements and contain at least
    ↪  one Street
178 assert ValidPublish {
179   all bp: BikePath | some bp.streets and bp.streets in Street
180 }
181
182
```

4.1. Examples

General Testing

First thing first, a general test was performed in order to check if the model works properly. As shown in the diagram, all entities are correctly linked and all requirements are met.

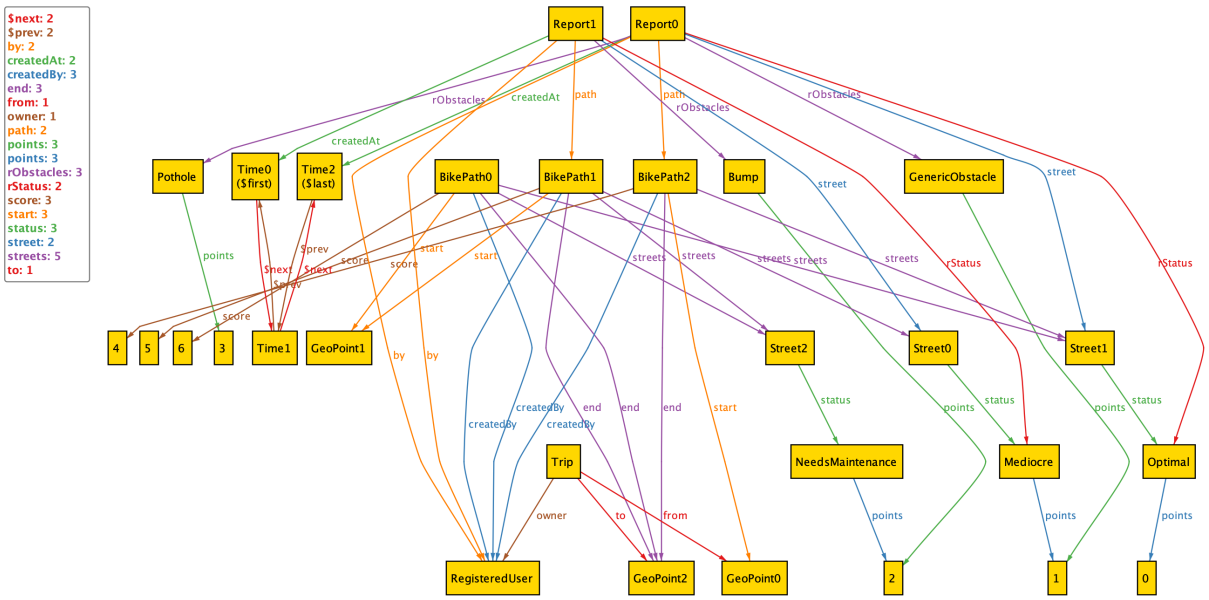


Figure 4.1: General Testing

4.1.1. Single report determines street status and path score

Here it was verified that it is a single report that determines street status and path score, by executing the following command:

```
run singleReportUpdates for 4
```

Which resulted in:

```
Executing "Run singleReportUpdates for 4"
  Actual scopes: 4 User, 4 RegisteredUser, 4 GeoPoint, exactly 4 Time, 3 Status, exactly 1 Optimal,
  Solver=sat4j Bitwidth=4 MaxSeq=4 SkolemDepth=1 Symmetry=20 Mode=batch
  7762 vars. 432 primary vars. 21711 clauses. 67ms.
  Instance found. Predicate is consistent. 21ms.
```

Figure 4.2: Demo 1: Result

And visually determined:

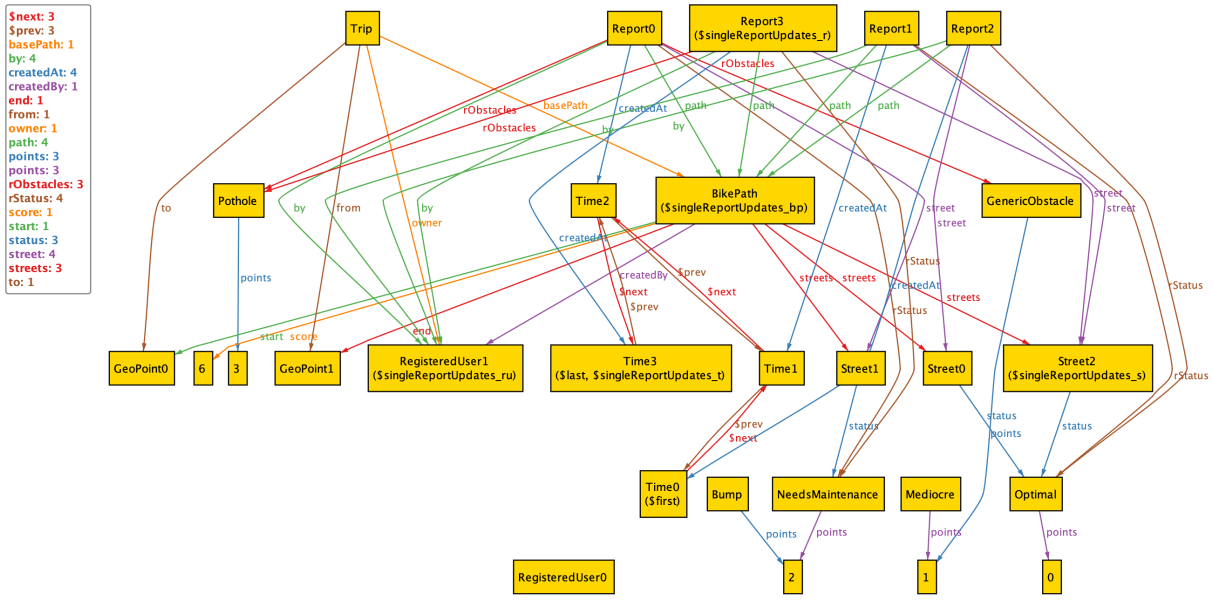


Figure 4.3: Demo 1: Tree

4.1.2. Freshness wins over older majority of reports

Here it was verified that freshness determines the report whose information is taken into consideration, by executing the following command:

```
run freshnessWins for 4
```

Which resulted in:

```
Executing "Run freshnessWins for 4"
  Actual scopes: 4 User, 4 RegisteredUser, 4 GeoPoint, exactly 4 Time, 3 Status, exactly 1 Optimal,
  Solver=sat4j Bitwidth=4 MaxSeq=4 SkolemDepth=1 Symmetry=20 Mode=batch
  8171 vars. 448 primary vars. 22629 clauses. 93ms.
  Instance found. Predicate is consistent. 27ms.
```

Figure 4.4: Demo 2: Result

And visually determined:

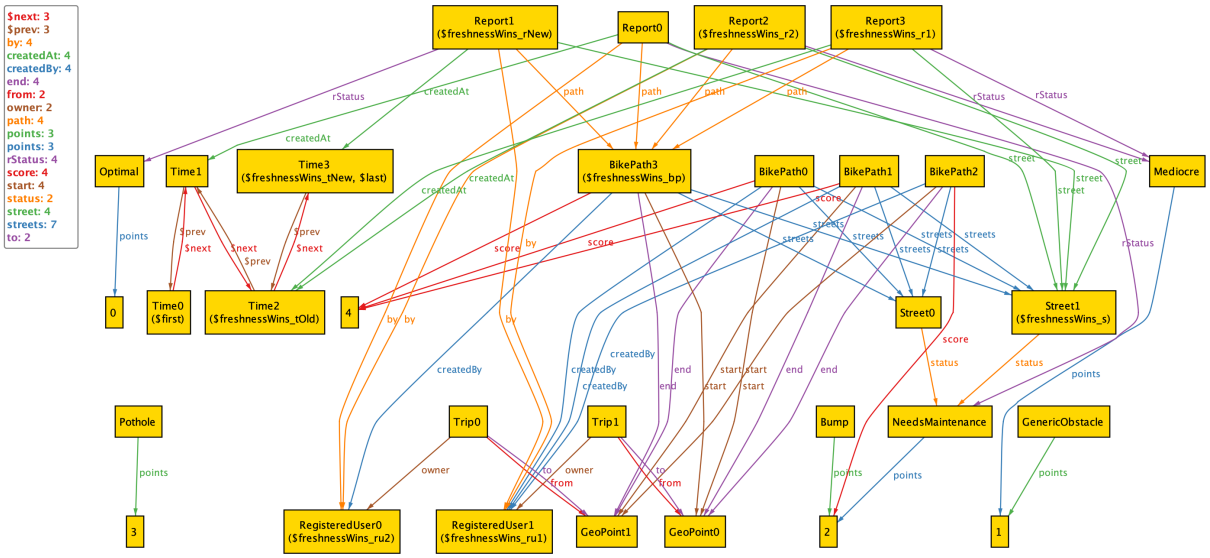


Figure 4.5: Demo 2: Tree

4.1.3. Majority voting wins among reports with the same freshness

Here it was verified that, having many report with same freshness, the report whose information is taken into consideration is selected by majority criteria (i.e., information with highest number of reports, with same highest freshness), by executing the following command:

```
run majorityVotingOnConflict for 4
```

Which resulted in:

```
Executing "Run majorityVotingOnConflict for 4"
  Actual scopes: 4 User, 4 RegisteredUser, 4 GeoPoint, exactly 4 Time, 3 Status, exactly 1 Optimal,
  Solver=sat4j Bitwidth=4 MaxSeq=4 SkolemDepth=1 Symmetry=20 Mode=batch
  8148 vars. 448 primary vars. 22626 clauses. 50ms.
  Instance found. Predicate is consistent. 13ms.
```

Figure 4.6: Demo 3: Result

And visually determined:

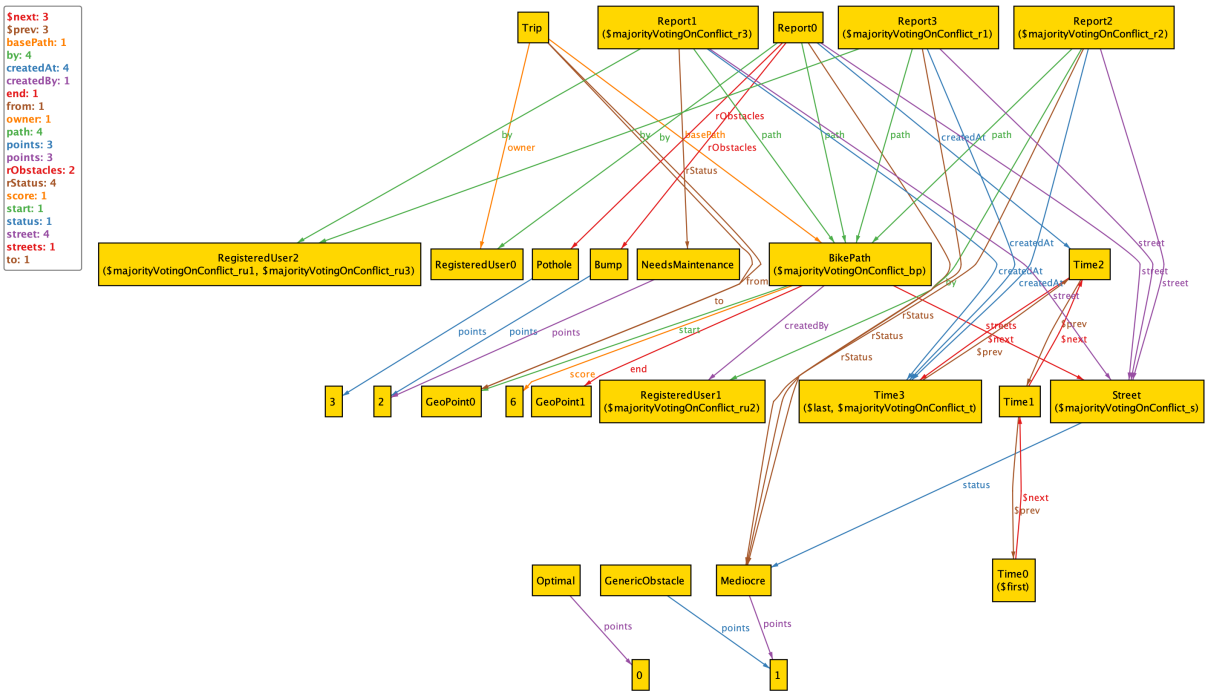


Figure 4.7: Demo 3: Tree

5 | Effort spent

Task	Simone Controguerra	Matteo Delogu	Gianluca Di Paola
Chapter 1	10 hours	10 hours	10 hours
Chapter 2	9 hours	5 hours	8 hours
Chapter 3	9 hours	10 hours	5 hours
Chapter 4	4 hours	7 hours	9 hours
Total Effort	32 hours	32 hours	32 hours

Table 5.1: Effort Spent by Each Group Member

6 | References

- Alloy official documentation: `https://alloytools.org/documentation.html`
- UML official specification: `https://www.omg.org/spec/UML/`
- Statistics about Italian daily bike users: `www.epicentro.iss.it/passi/dati/mobilita-attiva`